

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Elektro- und Informationstechnik

Bachelorarbeit

von

Thomas Mustermann

L^AT_EX im Griff

L^AT_EX under control

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Elektro- und Informationstechnik

Bachelorarbeit

von

Thomas Mustermann

L^AT_EX im Griff

L^AT_EX under control

Bearbeitungszeitraum: von 1. Oktober 2013
 bis 31. Januar 2014

1. Prüfer: Prof. Dr. Axel Maier
2. Prüfer: Prof. Dr. Bruno Müller

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname
der Studentin/des Studenten: **Mustermann, Thomas**

Studiengang: **Elektro- und Informationstechnik**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

L^AT_EX im Griff

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: **May 18, 2026**

Unterschrift:

Bachelorarbeit Zusammenfassung

Studentin/Student (Name, Vorname):	Mustermann, Thomas
Studiengang:	Elektro- und Informationstechnik
Aufgabensteller, Professor:	Prof. Dr. Axel Maier
Durchgeführt in (Firma/Behörde/Hochschule):	Renner-Verlag, Musterstadt
Betreuer in Firma/Behörde:	Dr. Karl Schulze
Ausgabedatum: 1. Oktober 2013	Abgabedatum: 31. Januar 2014

Titel:

L^AT_EX im Griff

Zusammenfassung:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Schlüsselwörter: L^AT_EX, Textsatz, Formeln, Graphiken

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Project Goal	2
1.3	Research Focus – Imitation Learning	3
2	Basics Classical Robotics	4
2.1	Erster Abschnitt von Kapitel 2	4
2.1.1	Ein Unterabschnitt	4
2.1.1.1	Ein Unterabschnitt	4
3	Basics Intelligent Robotics	5
3.1	Machine Learning Basics	5
3.1.1	Mathematical Notation of formulas	5
3.1.2	Glossary	5
3.1.3	Core Machine Learning Categories	6
3.2	Intelligent robots with Deep Learning (RL)	7
3.2.1	Reinforcement Learning	8
3.2.1.1	Mathematical description of the RL problem:	9
3.2.1.2	Knowledge Modality	11
3.2.2	Imitation Learning	12
3.2.2.1	Behavior Cloning (BC)	13
3.2.2.2	Learning from Videos / Observation	13
3.2.3	Summary IL & RL	14
4	Imitation Learning fundamentals	16
4.1	Why Imitation Learning?	16
4.1.1	Idea behind Imitation Learning:	16
4.1.2	Ways to instruct robots	17
4.2	Generalist Robots	20
4.2.1	Data Bottleneck in Robotics	21
4.2.2	Relevant Data Categories for Robotics	22
4.2.3	How to Address the Data Bottleneck	22
4.2.4	Reduce the amount of robot data	25
4.2.5	Challenges of Video Data	30
4.2.6	Other Paths to Generalist Robots.	32

5	Learning from Video Techniques	33
5.1	Extracting Foundational Knowledge from Internet Video	33
5.1.1	Video Encoders	34
5.1.1.1	Methods	35
5.1.2	Video Prediction Models	37
5.1.2.1	Methods	38
5.1.3	Video-to-Text Models.	40
5.1.3.1	Methods	41
5.1.4	Challenges and Future Directions	42
5.2	Video-to-Robot Transfer: Addressing Missing Action Labels and Distribution Shift	42
5.2.1	Categories of Alternative Action Representations	45
5.3	LfV and Reinforcement Learning	50
	Literaturverzeichnis	51
	Abbildungsverzeichnis	54
	Tabellenverzeichnis	55

Symbole, Formelzeichen und Einheiten

Chapter 1

Introduction

1.1 Motivation

Roboter werden zukünftig eine Schlüsselrolle in zahlreichen Anwendungsfeldern einnehmen. Bereits heute steigt der Einsatz von Robotersystemen in der industriellen Produktion kontinuierlich an, da diese insbesondere monotone, einfache und repetitive Aufgaben effizient und zuverlässig übernehmen können. Trotz der teilweise sehr hohen Anschaffungskosten, investieren immer mehr Unternehmen in die Integration von Roboter in Ihre Produktionslinien. Die Vorteile liegen dabei vor allem in der hohen Verfügbarkeit und Planbarkeit: Nach der Inbetriebnahme fallen nur noch geringe laufende Betriebskosten an, die sich primär auf Energieverbrauch in Form von Strom- und Wartungskosten beschränken. Darüber hinaus können Roboter unabhängig von der Arbeitszeit 24h, 7 Tage die Woche betrieben werden und liefern eine gleichbleibende Qualität. Dies ermöglicht eine hohe Prozessstabilität sowie eine signifikante Steigerung der Produktivität. Im Gegensatz zu menschlichen Arbeitskräften, werden Roboter nicht krank, nehmen nicht an einem Gewerkschaftsstreik teil und es müssen auch keine Sonderzahlungen in Form von Nacht-, Wochenend- und Feiertagszuschlägen geleistet werden. Grundsätzlich lassen sich Robotersysteme hinsichtlich ihrer Programmierung in zwei Kategorien einteilen. In der industriellen Praxis dominieren derzeit statisch programmierte Roboter, die für klar definierte Aufgaben in strukturierten Umgebungen eingesetzt werden. Sie haben feste Bewegungsabläufe und können sich nur in geringem Maße mithilfe integrierter intelligenter Sensoren an ihre Umgebung bzw. an die jeweilige Aufgabe anpassen. Solche Systeme sind besonders effizient in standardisierten Produktionsprozessen, stoßen jedoch an ihre Grenzen, sobald sich die Umgebung oder die Anforderungen ändern. Im Gegensatz dazu gewinnen intelligente Robotersysteme zunehmend an Bedeutung. Diese sind in der Lage, auf Veränderungen in ihrer Umgebung zu reagieren und ihr Verhalten adaptiv anzupassen. Insbesondere durch den Einsatz von Künstlicher Intelligenz eröffnen sich neue Möglichkeiten, komplexe und unstrukturierte Aufgaben zu bewältigen. Zentrale Herausforderungen bestehen jedoch hierbei in der robusten Ausführung vielfältiger Szenarien sowie in der Fähigkeit, auf neue Objekte, Umgebungen und Aufgaben zu generalisieren. Die heute eingesetzten intelligenten Robotersysteme, bzw.

deren zugrunde liegenden KI-Modelle sind oftmals stark auf ihre jeweilige Aufgabe und die Zielumgebung spezialisiert, bzw. trainiert worden. Aufgaben, die nicht im Trainingsdatensatz enthalten sind, können häufig nicht zuverlässig ausgeführt werden. Ebenso treten Probleme auf, wenn gelernte Aufgaben in grundlegend verschiedenen Umgebungen ausgeführt werden sollen. Der Grund hierfür liegt in der Art und Weise, wie die zugrunde liegenden KI-Modelle trainiert werden: Die Trainingsdaten sind in der Regel stark auf eine spezifische Aufgabe und Umgebung zugeschnitten. Eine Erhöhung der Datenvielfalt führt dabei häufig zu einem Zielkonflikt, da sie mit einer geringeren Genauigkeit bei der Ausführung einzelner Aufgaben einhergehen kann. Selbst wenn man ein großes generalisierendes Roboter KI-Modell entwickeln möchte, stellt der Mangel an geeigneten und vielfältigen Roboter-Trainingsdaten ein gravierendes Problem dar. Klassische, aufgabenspezifische Trainingsansätze stoßen hierbei aufgrund mangelnder Skalierbarkeit und Flexibilität an ihre Grenzen. Aktuelle Forschungsansätze verfolgen daher das Ziel, Robotern ein visuelles und semantisches Aufgabenverständnis zu vermitteln, sodass sie Handlungen aus hochdimensionalen Eingaben wie Videos oder Bildern ableiten können. Dadurch können neue Datenquellen für das Training des Roboters herangezogen werden. Mit einem visuellen und semantischen Verständnis können beispielsweise (menschliche) Videos von YouTube im Training verwendet werden. Diese beinhalten viele verschiedene Handlungen und Ausführungsformen verschiedenster Aufgaben, die ein generalisierendes Roboter KI-Modell brauchen würde. Imitation Learning ist ein vielversprechender Ansatz in diesem Kontext, der in dieser Arbeit genauer untersucht wird. Beim Imitation Learning geht es darum, den Roboter seine Aufgabe mithilfe eines (menschlichen) Demonstrationsvideos beizubringen (z.B. das Stapeln von Würfeln, oder das Greifen von Objekten). Beim traditionellen Ansatz muss ein Roboter auf die jeweilige Aufgabe mit viel Aufwand bzw. mit vielen Roboter trajectory Datensätzen trainiert werden.

1.2 Project Goal

Ziel dieser Arbeit ist es eine Einführung in das Thema Robotik mit Imitation Learning zu liefern. Die Arbeit gliedert sich in zwei Teile. Dabei werden zuerst die Grundlagen der Robotik erläutert und anschließend technisch in das Thema Intelligente Robotik mit dem Fokus Imitation Learning eingeführt. Abgerundet wird der erste Teil mit einem Literaturüberblick, über ausgewählte Imitation Learning Ansätze, die dort näher untersucht werden. Im zweiten Teil dieser Arbeit wird ein Open-Source Imitation Learning Modell experimentell auf einen realen Roboterarm implementiert. Die Erstmalige Inbetriebnahme des Roboters und das Setup, wie ein KI-Modell zur Ausführung auf den Roboter gebracht werden kann, gehört auch dazu. Ziel ist es, den Roboter zu befähigen, einfache Manipulationsaufgaben, beispielsweise das Stapeln von Würfeln, allein auf Basis menschlicher Demonstrationsvideos zu erlernen und auszuführen. Die Leistungsfähigkeit des vorgeschlagenen Ansatzes wird anschließend experimentell auf dem realen Robotersystem untersucht und evaluiert.

Beim Imitation Learning geht es darum, den Roboter seine Aufgabe mithilfe eines (menschlichen) Demonstrationsvideos beizubringen (z.B. das Stapeln von Würfeln,

oder das Greifen von Objekten). Beim traditionellen Ansatz muss ein Roboter auf die jeweilige Aufgabe mit viel Aufwand bzw. mit vielen Roboter trajectory Datensätzen trainiert werden. Beim Imitation Learning geht es darum, den Roboter seine Aufgabe mithilfe eines (menschlichen) Demonstrationsvideos beizubringen. Dabei nimmt eine Kamera ein Video von der jeweiligen Person auf, die eine bestimmte Aufgabe ausübt. Anschließend wird dem Roboter-KI-Modell das Demonstrationsvideo der Aufgabe gezeigt, welches wiederum den Roboter entsprechend steuert, um die jeweilige Aufgabe in seiner Umgebung auszuführen.

1.3 Research Focus – Imitation Learning

Chapter 2

Basics Classical Robotics

2.1 Erster Abschnitt von Kapitel 2

test test test test

2.1.1 Ein Unterabschnitt

2.1.1.1 Ein Unterabschnitt

jetzt gehts

Chapter 3

Basics Intelligent Robotics

3.1 Machine Learning Basics

3.1.1 Mathematical Notation of formulas

Capital letters are used for random variables, e.g. S represents the set of all states in which the robot can be and A represents the set of all possible actions it can perform.

Lowercase letters are used for specific values of these random variables, e.g. s stands for a specific robot state and a for a specific action that the robot performs. A lowercase letter is usually followed by a time index t , representing the specific action taken by the robot at time step t . In some cases the time index is omitted for simplicity, e.g. a instead of a_t and the next state or action is then denoted as s' or a' instead of s_{t+1} or a_{t+1} .

Capital letters in bold represents Matrices, e.g. the matrix $\mathbf{X}$.

Lowercase letters in bold are used for specific values of (random) variables that are vectors, e.g. $\mathbf{x}$ represents a specific vector.

3.1.2 Glossary

Agent & Policy:

An agent is an entity that learns to perform a specific behavior through interaction with its environment. In the context of this work, the agent corresponds to the robot, which has to execute a given action or task. The policy can be interpreted as the brain of the agent, determining which action to take next in a given situation in order to achieve the task objective. In our example, the policy controls the robot's motion and actions.

Formally, a policy can be represented as

$$\pi_{\theta}(a \mid s) \tag{3.1}$$

which defines a probability distribution over actions a , conditioned on the current state s . In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment.

In the classical meaning, the action a corresponds for example to a robot command, such as moving the gripper to a target position (x, y, z) or setting a joint angle to a specific value with a defined velocity. The state s represents the agent's observation, which may include, for example, a sequence of recent camera images (e.g., the last four frames) as well as additional sensor inputs such as proprioceptive signals.

Robot Trajectory:

A robot trajectory describes how the robot moves from a starting state to a target state, including position, orientation and, if applicable, speeds, accelerations, and forces acting on the robot. A robot trajectory is a temporal sequence of actions and the resulting robot states.

Annotation:

Annotation refers to the process of labeling data. Data is provided with additional information. For example, class affiliations can be determined, target objects can be marked in images, or states and actions can be marked in robot trajectories. Annotated data, or labeled data, form the basis for supervised learning approaches.

Annotating refers to the process of labeling data, in which data is enriched with additional information. For example, this may include assigning class labels, marking target objects in images, or labeling states and actions within robot trajectories. Annotated, or labeled, data forms the foundation for supervised learning approaches.

Foundation Model vs. Multi-Model Model

3.1.3 Core Machine Learning Categories

There are various approaches to training an AI model for a robot. These involve methods from the four core machine learning categories, which can also be combined with one another:

Supervised Learning:

In supervised learning, a model is trained using labeled training data. This means that, for the input data, the output data to be predicted are known. The goal is to train a model that correctly predicts the output data and makes reliable predictions on unseen data.

Unsupervised Learning:

In contrast to supervised learning, unsupervised learning uses unlabeled data for model training. The model tries to recognize structures, patterns or relationships in

the data on its own.

Semi Supervised Learning:

Semi-supervised learning is a combination of supervised and unsupervised learning. In this approach, a model is trained using a small amount of labeled data and a large amount of unlabeled data. The goal is to leverage the advantages of both approaches, particularly when annotating data is time-consuming or cost-intensive. This is relevant in robotics, as the data used in that field is often difficult to label.

Reinforcement Learning:

Reinforcement learning (RL) differs fundamentally from the previously mentioned approaches. Instead of learning from fixed datasets, an agent actively interacts with its environment and tries to find out an optimal strategy to reach a defined goal. Through its actions, it influences the state of the environment and subsequently receives feedback in the form of rewards or penalties, see Figure 3.1 for a short overview. In the context of robotics, the robot is the agent. The objective is to learn a strategy (= policy) that maximizes the cumulative reward over time for the given task.

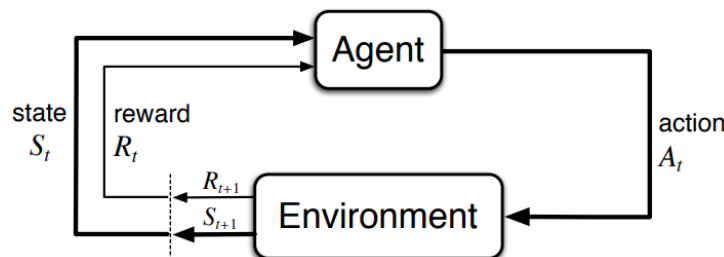


Figure 3.1: Base concept of Reinforcement Learning

The agent proceeds in its environment according to the trial-and-error principle to learn an optimal policy. Safety is a critical concern during training, as the agent may execute unpredictable actions while exploring the state space. For this reason, simulation environments are a central component of training and are frequently used for pre-training the policy. The policy is then subsequently often fine-tuned in the real-world environment. Because the agent learns through trial and error, more efficient, unknown, and creative solutions can be found that a human might not have come up with. See section 3.2.1 for more details.

3.2 Intelligent robots with Deep Learning (RL)

In general, modern robot training distinguishes between two main approaches: imitation learning and reinforcement learning. Each approach uses a different strategy for training the robot model. In Figure 3.2 an overview of the two approaches is given. The following sections explain the two approaches in more detail and provide a mathematical description of the underlying problem.

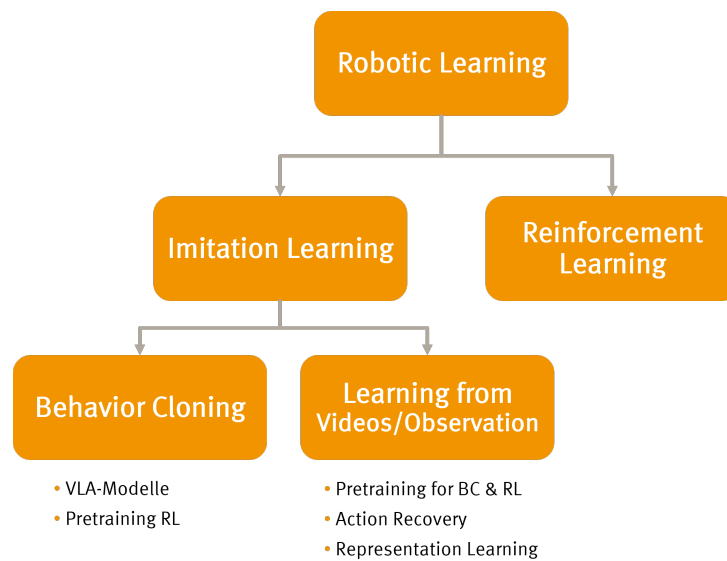


Figure 3.2: Overview Robotic Learning

3.2.1 Reinforcement Learning

Reinforcement learning (RL) is both a fundamental discipline of machine learning and a distinct approach within Robot Learning. The robot learns a policy through trial and error using rewards provided from the given task and the environment. The actions performed by the agent are evaluated with a reward score, and this score is then used to gradually optimize the policy.

Key components of the RL problem The following section explains the key concepts in RL related to robotics, see also figure 3.1 for a general graphical overview.

- **Agent:** The agent is the entity that learns to perform a specific behavior through interaction with its environment. In the context of this work, the agent corresponds to the robot, which has to execute a given action or task.
- **Environment:** The environment is the setting in which the agent (the robot) operates. Simply put, the environment is the robot's workplace, where it is deployed. In addition to the robot itself, it includes all relevant objects with which the robot interacts. The environment can be either the real world or a simulated environment in which the robot is trained.
- **Robot Actions:** The Robot actions are simply the control commands the robot can send to its actuators, e.g. open or close the gripper, move the arm to position X, Y, Z , or the joint angles of the individual robot axes, etc.
- **Reward:** The reward is a numerical value that indicates how well the robot is performing in relation to the task objective. For example, if the robot successfully picks up an object, it might receive a positive reward, while if it fails or drops the object, it might receive a negative reward. The goal of Reinforcement Learning is to learn a policy that maximizes the cumulative reward over time.
- **Robot State:** state is a complete description of the situation in the environment

at a particular point in time. The robot state represents the current situation of the robot and its environment. It can include various types of information, such as the robot's position, orientation, joint angles, sensor readings, and even visual input from cameras [35]. The state is crucial for the robot to make informed decisions about which actions to take.

- **Observation:** An observation is the information that the robot receives from its sensors at a given time step. It may not provide a complete picture of the environment, and thus may not fully represent the true robot state. In our case, the observation is mostly equivalent to the state [35].
- **Policy:** The policy can be interpreted as the brain of the agent. The policy determines which action should be taken next when the agent is in a particular state in order to achieve the task objective. The Policy controls the robot's motion and actions.

3.2.1.1 Mathematical description of the RL problem:

Die RL Modularitäten werden wie folgt dargestellt:

- Action: a
- Reward: r
- State: s
- Observation: o
- Policy: π
- Time step: t , e.g. a_t is the action taken at time step t , a_{t+1} is the action taken at the next time step, etc.
- s' : The next state after taking action a in state s , i.e. $s' = s_{t+1}$, $s = s_t$.

Markov Decision Process

Reinforcement Learning is based on the Markov decision process (MDP). A MDP is a stochastic decision-making process that formally models the transition probabilities from a state s to a subsequent state s' provided that action a is performed, expressed mathematically as $P(s'|s, a)$ [30, 19].

The objective is to learn a policy $\pi(a|s)$ that maximizes the expected long-term cumulative reward, defined as $\mathbb{E} [\sum_{t=0}^{\infty} \gamma^t r_t]$. The discount factor $\gamma \in [0, 1]$ determines the importance of future rewards compared to immediate rewards. A value of γ close to 0 makes the agent focus more on immediate rewards, while a value close to 1 encourages the agent to consider long-term rewards more heavily [30, 10].

An MDP also defines a reward function that specifies the reward r an agent receives when taking action a in state s and transitioning to a subsequent state s' . This is commonly expressed as $R_a(s, s')$ [30, 19].

The **Markov property** says that all information from the past is contained in the current state, mathematically expressed as $P(S_{t+1}|S_t) = P(S_{t+1}|S_1, S_2, \dots, S_t)$. In other words, the transition probabilities depend only on the current state and not on the history of previous states [10].

In practice, the Markov property is often not strictly satisfied. For example, a single image captured by a camera does not provide sufficient information to infer the dynamics of the environment. From one image alone, it is not possible to determine whether a car is moving forward at 100 km/h or 10 km/h, reversing, or remaining stationary. However, such information is crucial for making correct decisions in applications like autonomous driving [29]. Therefore, in practice, only sufficiently informative state representations are often used because the ideal Markov property is not strictly fulfilled. This is often achieved by incorporating a history of observations, such as a sequence of the last five camera frames, to better capture the underlying dynamics of the environment [19].

Training process:

Zu Beginn des Trainingsprozesses ist die Policy in der Regel zufällig, was bedeutet, dass der Roboter zunächst zufällige Bewegungen ausführt. Durch Ausprobieren versucht der Roboter nun Aktionen zu finden, die mit einer hohen Belohnung einhergehen. Diese Aktionen werden anschließend von der Policy höher gewichtet, was bedeutet, dass der Roboter in Zukunft eher dazu neigt, diese Aktionen auszuführen. Für eine erfolgreiche Lernphase ist es wichtig, dass der Roboter eine gute Balance zwischen Exploration (Ausprobieren neuer Aktionen im jeweiligen Zustand) und Exploitation (Nutzung bereits bekannter, erfolgreicher Aktionen im jeweiligen Zustand) findet. Auf die Exploration wird zu Beginn des Trainingsprozesses mehr Wert gelegt, um die Umgebung besser kennenzulernen und die effizientesten Aktionen im jeweiligen Roboter State für den Erfolg der Aufgabe zu finden. Im Laufe der Zeit, wenn der Roboter mehr über die Umgebung und die Belohnungsstruktur gelernt hat, wird die Exploitation wichtiger, um die bereits gefundenen erfolgreichen Aktionen häufiger auszuführen und so die Gesamtleistung, die am Aufgabenerfolg gemessen wird zu verbessern. Es existieren zahlreiche Methoden und Algorithmen, wie die Exploration und Exploitation Phasen effektiv gesteuert werden können. Beispielalgorithmen sind ϵ -greedy, Upper Confidence Bound (UCB), Thompson Sampling. Im Rahmen der Arbeit wird aber nicht näher darauf eingegangen.

Durch die Explorationsphase wird erst spät der Erfolg des gewählten RL-Ansatzes sichtbar, anders als beim Klassischen Deep Learning, siehe Abbildung 3.3.



Figure 3.3: Training Progress in Deep Learning and Reinforcement Learning

3.2.1.2 Knowledge Modality

An RL Knowledge Modality (KM) is some function learned from data that represents specific types of RL-related knowledge. KMs can be learned through trial-and-error principles or from various data sources, like video datasets, and can capture different aspects of the RL problem.

Knowledge Modalities (KMs) from Video

- **Policy:** A Policy $\pi(a_t|s_t)$ is a mapping from states to actions and defines the behavior of the agent that is in a particular state. $\pi(a_t|s_t)$ is the probability of taking action a_t given that the agent is in state s_t . t denotes the time step. In other words, the policy specifies how likely the agent is to select a particular action given its current observation of the environment. It can be learned from video data by observing the actions taken in different states and learning to predict those actions based on the visual input.
- **Value functions:** Value functions estimate the expected future return (cumulative reward) for a given state or state-action pair. They can be learned from video data by observing the outcomes of actions and learning to predict the expected future rewards based on the visual input.
 - **State-value function (V-function):** A V-function $V_\pi(s_t)$ is a mapping from states to the expected future return when acting under a particular policy π .

Intuition: "How good is it to be a certain state, if I follow my policy π ?"

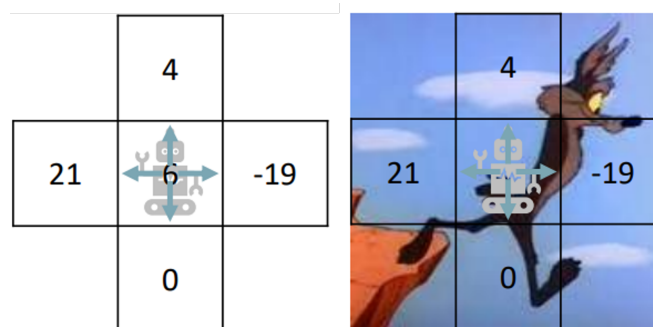


Figure 3.4: State-Value Function

- **State-action value function (Q-function):**

A Q-function $Q_\pi(s_t, a_t)$ is a mapping from state-action pairs to the expected future return when acting under a particular policy π .

Intuition: "How good is this specific action here, if I continue behaving like my policy π ?"

- **V-function vs. Q-function:** Bei der V-function weiß ich nur, wie gut es ist, in einem bestimmten Zustand zu sein, wenn ich nach meiner Policy π agiere, aber nicht, wie gut eine konkrete Aktion in diesem Zustand ist. Bei

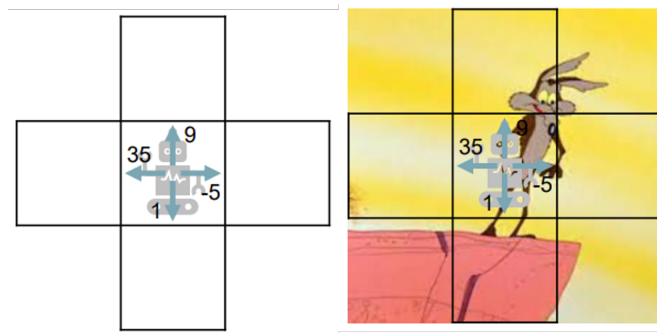


Figure 3.5: State-Action Value Function

der Q-function hingegen weiß ich bereits, wie gut die konkrete Aktion a_t im Zustand s_t ist, wenn ich weiterhin nach meiner Policy π agiere. Siehe dazu die Abbildung 3.4 und 3.5.

- **Dynamics model:** A Dynamic model $p(s_{t+1}|s_t, a_t)$ predicts the next state s_{t+1} given the current state s_t and action a_t . Video prediction models can be used as robot dynamics models. By learning to predict future video frames based on current observations and actions, the model can capture the underlying dynamics of the environment. This allows the robot to anticipate the consequences of its actions and make informed decisions based on predicted future states.
- **Reward model:** A reward function $r(s_t, a_t)$ defines the reward received by the agent for taking action a_t in state s_t . It can be learned from video data by observing the outcomes of actions and inferring the underlying reward structure that motivates those actions. For example, if a video shows a robot successfully picking up an object, the reward function can learn to assign a positive reward to that action in similar states.

3.2.2 Imitation Learning

Imitation Learning (IL) or Learning from Demonstration is a general term for methods in which the agent derives or learns its behavior from expert demonstrations.

Imitation learning makes it possible to use "in the wild" videos—such as those found in great variety on the internet, for example on platforms like YouTube — to train AI models.

The demonstration videos can show a robot, a human, or even a loose sequence of actions, both in the real world or in a simulation environment, performing a specific task. However, videos without a clearly defined objective or with unknown activity can also be used.

This allows models to be trained to capture the visual and semantic relationships, as well as the physical dynamics of the real world. This, in turn, is a key prerequisite for the development of a generalist policy. Imitation learning can be divided into Behavior Cloning and Learning from Videos/Observation.

3.2.2.1 Behavior Cloning (BC)

Behavior Cloning (BC) is a fundamental approach within imitation learning, where a policy is learned directly from **annotated expert demonstrations**. The training data consists of state–action pairs (s_t, a_t) , and the problem is formulated as a standard supervised learning task.

Conceptually, Behavior Cloning can be understood as a form of direct imitation (“direct copying”) of expert behavior, without explicitly modeling the underlying system dynamics or task objectives. Due to its simplicity and scalability, BC is one of the most widely used methods in robot learning from datasets.

BC is used to train Vision Language Action models and for pre-training models used in Reinforcement Learning, see Section XXX

Definition:

Behavior Cloning (BC) learns a policy by supervised learning on expert demonstrations:

$$\pi_{\theta}(a_t \mid s_t) \quad (3.2)$$

where:

- s_t : State / Observation (images, proprioception, etc.) at time t
- a_t : Expert action at time t

The Policy $\pi_{\theta}(a \mid s)$ is optimized by minimizing the prediction error between the actions generated by the model and the corresponding expert actions.

$$\min_{\theta} \mathbb{E}_{(s,a) \sim \mathcal{D}} [\mathcal{L}(\pi_{\theta}(s), a)] \quad (3.3)$$

where:

- $\pi_{\theta}(s)$: The action your robot’s policy predicts given state s .
- a : The actual action the expert took in that same state.
- $\mathcal{L}$: Usually Mean Squared Error (MSE) for continuous control or Cross-Entropy for discrete actions.
- $\mathbb{E}_{(s,a) \sim \mathcal{D}}[\cdot]$: Take all the state-action pairs (s, a) inside the expert dataset $\mathcal{D}$, and calculate the average value of the expression inside the brackets.

3.2.2.2 Learning from Videos / Observation

Learning from Videos (LfV), also referred to as Learning from Observation (LfO), is an Imitation Learning approach that relies solely on **video demonstrations without explicit action annotations**. In contrast to Behavior Cloning, where state–action pairs are directly available, LfV aims to infer the underlying actions or high-level intent purely from perceptual data, i.e., mapping observations to actions ($s \rightarrow a$). It often uses *Representation Learning*, to learn and discover the most important (unknown)

underlying data characteristics (features) by transforming and reducing input in a fully data-driven and automatic manner.

A central challenge in LfV is the **inference problem**: determining which actions correspond to the observed behavior in the video. Since no action labels are provided, additional methods and techniques are required to recover or approximate the missing action information. See chapter XXX for more information.

Due to this property, LfV is typically categorized as a **weakly supervised or self-supervised imitation learning** approach. Its main advantage lies in scalability, as it enables leveraging large amounts of unannotated video data, including internet videos or human demonstrations, without the need for costly data annotation. Furthermore, additional actions can be derived from annotated video data too.

In robotics, Learning from Videos is commonly used as a **pretraining step**. The learned representations capture visual, semantic, and dynamical aspects of the environment, which can later be refined using methods such as Behavior Cloning with annotated data or in Reinforcement Learning. This combination allows models to first learn the real worlds dynamics from large-scale video data and subsequently specialize on task-specific robot demonstrations.

Definition:

Learning from Videos (LfV) learns to infer actions from observations in a weakly supervised or self-supervised manner.

$$\hat{a}_t = f(s_t, s_{t+1}) \quad (3.4)$$

where:

- $\hat{a}_t$: The "latent" or predicted action.
- s_t : The current state/video frame.
- s_{t+1} : The next state/video frame.
- f : The inverse dynamics function (often a neural network) that maps state transitions to actions.

3.2.3 Summary IL & RL

Reinforcement learning is a powerful approach for learning complex tasks, as it enables agents to improve their behavior through interaction with the environment. However, it can be sample inefficient and may require a large number of interactions to learn an effective policy, especially in high-dimensional state and action spaces. Therefore, it is often combined with other learning paradigms, such as supervised learning or imitation learning, in order to leverage their complementary strengths and improve overall performance.

Imitation Learning generally refers to a family of methods and techniques that enable learning from demonstrations. Two important approaches in this context are Behavior

Cloning (BC) and Learning from Videos (LfV), which differ primarily in the type of data they utilize. BC relies on annotated demonstrations, where state-action pairs are available, and learns a policy directly via supervised learning. In contrast, LfV aims to infer actions from unannotated video observations, where only visual information is provided.

Methods from the LfV paradigm can be used for Action Recovery, making unannotated video data applicable for Behavior Cloning by recovering or estimating the underlying actions. Furthermore, such methods can be employed to pretrain models for BC or Reinforcement Learning, for example by learning expressive state and action representations through Representation Learning.

Behavior Cloning is also widely used for training Vision-Language-Action (VLA) models and for pretraining Reinforcement Learning Knowledge Modalities, for instance by learning components like value functions or reward models. In practice, BC and LfV are often combined to exploit the advantages of both approaches, enabling the use of large-scale unannotated data while maintaining the efficiency of supervised policy learning.

Chapter 4

Imitation Learning fundamentals

Was sollte noch erklärt werden?

Online Learning / Offline Learning

Multi-Modal und Foundation Models

4.1 Why Imitation Learning?

4.1.1 Idea behind Imitation Learning:

When we want to learn a new skill, we often turn to How-To videos. We learn the new skill through speech instructions and visual observations in the video.

Advantages of Imitation Learning:

Visual IL ...

- is ideal for teaching tasks that are difficult or impossible to describe in words.
- allows robots to infer tasks from expert demonstrations in different environments than the robot.
- can be more data efficient than reinforcement learning, especially in high-dimensional state spaces, like a complex robot environment.
- can leverage large datasets of human demonstrations, which are often easier to collect than expert demonstrations in the specific domain.

Problems in RL: A Task in RL is commonly defined through a reward function, which assigns a high reward to desirable behavior and a low reward to undesirable behavior. However, designing good reward functions is challenging for every task, and many methods based on computer vision rely on annotated demonstrations in every new environment.

IL allows us to learn complex behaviors that are difficult to specify through a reward function, or it can help us to define a good adaptive reward function for RL.

Robotic specific problems: Gap between videos of humans and robots performing similar tasks Gap between simulation and Real-World Simulations & Trial and Error in real World possible Limitations: time and computational consuming, Safety?

4.1.2 Ways to instruct robots

This section discusses the various ways in which the robot can be taught its task and how the interaction between humans and robots can be designed.

Scene Understanding / Intent Inference

The robot derives its task solely from the environment and the objects present in it [12].

One Policy, Multiple Tasks: In this setting, a single policy is typically specialized on a finite set of task instances, as they must be explicitly trained for those tasks. Tasks that are not represented or underrepresented in the training dataset, can therefore often not be executed reliably [4, 16]. For example, if the policy has been trained on multiple manipulation tasks and one of them involves pouring liquid from a teapot into a cup, the policy will execute this behavior when a matching scene is observed during inference [12]. However, generalization to other objects or situations is often limited. Using different teapots or cups may not be problematic as long as the relevant visual features are recognized. In contrast, performance typically decreases under substantial domain shift (significantly different objects or scenes) [12]. If a corresponding pouring behavior from a pot was not included during training, the policy may fail to transfer the action to that new object category [4, 12]. A major limitation of this approach is task ambiguity caused by overlapping object context across different behaviors. If the same object types, such as a teapot and a cup, appear in both a pick-and-place task and a pouring task, the policy may not be able to disambiguate the correct action target based on scene context alone [31].

One Policy, One Task: An alternative approach is to train a separate policy for each individual task, which is then selected explicitly during inference. The task-specific policy selection can be performed either manually by a human or automatically by a separate AI classification model that analyzes the scene and determines the appropriate task-specific policy [8]. For example, one policy could be trained for “pouring”, another for “opening a drawer”, and another for “setting up an overturned object”. This modular design is closely related to task-aware expert routing and decoupled planning-and-execution frameworks in robotic manipulation [8].

This decoupling enables training each policy on significantly larger and more diverse datasets containing varied object appearances and environmental conditions, while keeping the core task semantics fixed [28]. Consequently, these specialized policies often support more robust generalization across novel object instances and environmental variations compared to the One Policy, Multiple Tasks paradigm. [2, 22, 31]

Language instructions:

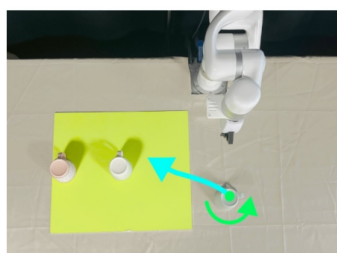
For humans is the most comfortable and natural way to instruct the robot with voice commands. This is very useful for simple tasks that are easy to describe. But complex tasks, where execution must follow exact instructions, are sometimes difficult to describe in words. For example, how would you teach a humanoid robot to decorate tables exactly like a sample table, especially when various decorations need to be assembled in a complex arrangement? Furthermore, ambiguities may arise, and the same instruction may be interpreted differently in different linguistic contexts. For example, the word “open” is used for: open drawer, open cabinet, open container with lid or open jar with screw cap. Although the same verb is used here, each interaction requires a completely different motor control.

A big advantage to command the robot with voice commands is that it enables an interactive dialogue between humans and the robot. If the robot does not understand the instructions, it can ask follow-up questions to clarify any misunderstandings. This helps minimize the problems associated with voice commands.

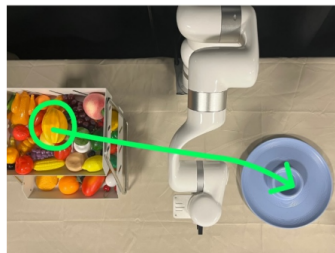
Image instructions:

Robots can also be instructed using visual inputs such as images. Compared to language, image-based instructions provide a more concrete and unambiguous specification of spatial arrangements, object relations, and desired outcomes.

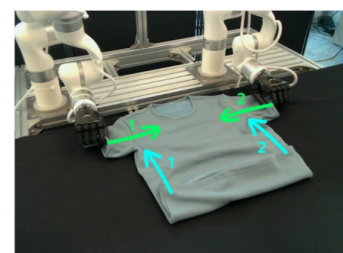
A common approach is the use of *goal images*, where the robot is given a visual representation of the desired final state. In this setting, the task is formulated as a goal-conditioned problem: the robot observes the current state and acts such that it minimizes the difference to the goal image. This is often referred to as goal-conditioned behavior. However, this approach does not explicitly specify *how* the task should be executed, only *what* the final configuration should look like. But some works have addressed this issue with the usage of annotations in the image of the initial state to specify the task execution. For example [2] highlights the relevant objects and regions with circles and arrows to visualize the task execution in the image.



(a) Task: Rotate cup and place it on table



(b) Task: Put fruit into bowl



(c) Task: Fold T-shirt

Figure 4.1: Image-based instructions for task execution [2].

Another way to address this limitation, *subgoal images* or intermediate states can be provided [25]. These guide the robot through a sequence of visually defined steps, effectively decomposing complex tasks into simpler stages. This is particularly useful for long-horizon or multi-step manipulation tasks.

A key advantage of image instructions is their ability to capture fine-grained spatial details that are difficult to describe in language, such as precise object placement, orientation or symmetry alignments. However, they also introduce challenges related to perception, such as viewpoint variation, occlusions of objects in the real environment, or the use of in-the-wild goal-images with varying lighting and background conditions.

Video-Based Instructions:

Demonstration videos are a powerful way to instruct robots, as they contain rich visual and temporal information. In contrast to single images, videos provide not only the desired goal state of a task but also the sequence of intermediate steps required to reach that goal.

In this setting, the task is typically formulated as an imitation learning problem, where the robot learns a policy that maps visual observations to actions by imitating the demonstrated trajectory. The temporal structure of the video enables the robot to infer *how* a task should be executed, rather than only *what* the final outcome should look like. Depending on the setup, this may also require inferring latent actions from visual observations alone.

Ideally, the robot model should be able to understand its task from in-the-wild demonstration videos of both robots and humans. In-the-wild videos refer to video data collected in unconstrained, real-world environments, without controlled settings, standardized viewpoints, or curated conditions. Such data exhibits large variability in lighting, backgrounds, and camera motion, making it significantly more challenging for perception and learning compared to controlled laboratory data.

Human Video Demonstrations:

- **Data abundance:** In this paradigm, the robot learns from videos of humans performing a task. This is particularly appealing, as large amounts of human demonstration data are readily available (e.g. online videos).
- **Embodiment gap:** A core challenge is the mismatch between human and robot bodies. Human motions and kinematics differ significantly from those of robots. Therefore, the robot must learn to map observed human actions to its own action space.
- **Suboptimal demonstrations:** Another key challenge of human video demonstrations is that they do not necessarily represent optimal task executions. Demonstrations often reflect individual preferences, inefficiencies, or context-specific behaviors, resulting in suboptimal trajectories. In many cases, a video captures only a single possible way of solving the task, without providing information about alternative or more efficient strategies. However, the objective for the robot is typically to perform tasks efficiently and robustly. Therefore, the learned policy must be able to generalize beyond the demonstrated behavior and, if necessary, improve upon it, rather than directly imitating potentially suboptimal actions.

Robot Video Demonstrations:

- **No embodiment gap:** The demonstrations are performed by the robot itself or by another robot with similar embodiment. This avoids the human-robot embodiment gap and provides more directly transferable data.
- **Cost and effort:** However, collecting such demonstrations is often expensive and time-consuming, as it requires access to robotic systems and typically controlled environments. In addition, purely visual robot demonstrations may still lack explicit action labels. This means that the robot must still infer the underlying actions and intentions from the visual data.
- **Value of robot demos:** Despite these challenges, robot video demonstrations can provide valuable data for learning, as they are more closely aligned with the robot's own capabilities and constraints compared to human demonstrations.

Teleoperation:

- **Direct data collection:** A common method to generate high-quality robot demonstrations is teleoperation, where a human operator directly controls the robot, e.g., via a joystick or a dedicated control interface.
- **Precision and safety:** In contrast to passive video demonstrations, teleoperation provides access to both the visual observations and the corresponding action and trajectory data. As a result, the demonstrated actions are already expressed in the robot's action space, which significantly simplifies reproducing the expected behavior. Such demonstrations are particularly important for fine-grained manipulation tasks that require precise control and accurate timing.
- **Scalability issues:** However, teleoperation may require specialized hardware (controllers, sometimes motion-capture or VR setups) and does not scale as easily as collecting passive video data. Thus, collecting large teleoperation datasets can be time-consuming and is typically done in lab settings only.

4.2 Generalist Robots

The ultimate goal of robotics is to develop a `generalist` robot. According to McCarthy et al. [17], such a robot should be able to perform a wide variety of physical tasks in unstructured, unseen real-world environments. It must combine high-level reasoning and planning abilities with low-level physical skills, such as dexterous manipulation. Furthermore, it should be capable of adapting to unforeseen situations and operate under imperfect and partial observations, for example through visual and tactile sensing. In summary, a general-purpose robot is capable of adapting to different situations and learning new skills just like humans [33].

Such a robot would be highly useful across a wide range of real-world applications. In `industrial` settings, this would enable robots to be deployed flexibly across different environments and tasks, without the need for task-specific or environment-specific programming or retraining. As a result, robots could also be utilized for spontaneous small, short-term tasks, where automation is currently not economically viable due to

high setup costs and effort. This increased flexibility would lead to higher utilization rates and, consequently, reduced long-term operational costs [33].

In addition, general-purpose robots could play an important role in household settings. They could assist with everyday tasks such as tidy up rooms, (un)loading dishwashers, setting tables, or acting as general assistants for example helping to carry objects.

In healthcare and assistive contexts, such robots could support individuals with limited mobility by helping them perform daily activities, such as standing up, sitting down, opening a door, or transporting objects. This highlights the potential of generalizing robotic systems to improve both efficiency in industry and quality of life in everyday and care settings [33].

Ethical concerns: Ethical concerns regarding the development of generalist robots should not be underestimated and must not be ignored. In the future, robots are likely to take over many tasks, particularly those that are repetitive and monotonous, which may lead to the displacement of certain jobs. Whether this will result in large-scale unemployment due to automation remains a subject of ongoing debate.

Furthermore, once the technology has sufficiently matured, such robots could also be used for military purposes. While this could reduce the risks to human soldiers, it also raises serious ethical concerns, as it makes forms of automated or autonomous warfare increasingly conceivable.

It is therefore essential that the development and deployment of generalist robots are carried out responsibly and in accordance with ethical guidelines, in order to minimize potential negative impacts on society.

4.2.1 Data Bottleneck in Robotics

A central question in robotics is how to train a generalist robot capable of performing a wide range of tasks in unstructured and previously unseen real-world environments?

Usage of large-scale datasets. Progress in related machine learning domains—such as Natural Language Processing [1], Computer Vision [3], and video generation [7]—suggests that large-scale datasets and model capacity are key drivers of performance and generalization capabilities. Training high-capacity models with expressive architectures on diverse, extensive internet-scale datasets has led to the emergence of robust representations and improved out-of-distribution performance [34]. Empirically, these improvements often follow predictable scaling laws, where performance improves as more data, model size, and compute are used [13]. Recent work suggests that similar scaling behavior may also apply to robotics and imitation learning. For example, studies show that robots generalize better when they are trained on more diverse training data, including variations in environments, objects, and tasks [15, 9]. Moreover, large-scale imitation learning approaches based on behavioral cloning have demonstrated that training a single policy across many tasks can yield substantial gains in robustness and generalization. For example, transformer-based robot policies such as RT-1 and RT-2 leverage large, heterogeneous datasets to achieve zero-shot generalization to novel tasks and instructions [5, 6].

Data bottleneck. In contrast to vision and language domains, robotics is fundamentally constrained by the cost and complexity of data collection, limiting the availability of large, diverse, and high-quality datasets. High-quality demonstrations require physical real world interaction, making them time-consuming to acquire, and are often limited in diversity due to hardware and safety constraints [33]. As a result, available datasets are usually smaller and less diverse. Consequently, this data scarcity represents a major bottleneck for achieving similar levels of scalability and generalization in robot learning, and remains a central challenge in the development of general-purpose robotic systems [33].

4.2.2 Relevant Data Categories for Robotics

4.2.3 How to Address the Data Bottleneck

How can we address this data bottleneck in robotics? Before discussing specific approaches, it is important to clarify which data sources are relevant for robot learning and how such data can be acquired.

- **Real world data** Real-world robot data is obtained through physical interaction between the robot and its environment and represents a high-quality source for robot learning. Through direct interaction, trajectories and sensor measurements can be recorded under real operating conditions, capturing effects such as sensor noise and environment variability that are difficult to model accurately in simulation.

Teleoperation. A common approach to collecting such data is human teleoperation, where an operator controls the robot to perform tasks and generate demonstration trajectories [36, 129, 195]. This type of data has been widely used in imitation learning and has contributed to the development of early robot foundation models [36, 264]. Teleoperation can be realized through various control interfaces, including joysticks, motion-capture systems, or virtual reality setups. Teleoperation enables precise control and is particularly suitable for collecting demonstrations of complex manipulation tasks that require fine-grained movements and accurate timing. However, it is time-consuming and often requires specialized hardware, which limits scalability. To mitigate this, datasets are increasingly aggregated across multiple research institutions, improving both scale and diversity [195].

Data collection improvements. To further address the limitations of manual data collection, recent work has explored automated or semi-automated data acquisition methods, where robots collect data with reduced human intervention [35, 5, 304]. In parallel, advances in robotic infrastructure, particularly in industrial settings, indicate that large-scale data collection in real-world environments is becoming increasingly feasible [251, 115].

Despite these developments, real-world datasets remain significantly smaller and less diverse compared to internet-scale datasets used in vision and language

domains. This limitation continues to pose a challenge for scaling robot learning methods and achieving robust generalization.

- Simulated data

Simulatedly Generated Demonstration Data. Simulation enables the generation of robot trajectories in virtual environments without the cost, safety risks, and hardware limitations associated with real-world experiments. As a result, simulated demonstrations can be used in scenarios where collecting physical robot data would be impractical, expensive, or unsafe [336, 127, 190]. These generated trajectories are commonly used in imitation learning especially in Behavior Cloning, where policies are trained on demonstration.

Expert demonstrations in simulation are typically generated by executing a high-performing controller or policy in the simulated environment, such as a hand-crafted controller, a planning-based system, or a policy obtained via reinforcement learning.

- **hand-crafted controllers:** Hand-crafted controllers are based on classical control methods such as PD/PID controllers, Linear Quadratic Regulators (LQR), or Model Predictive Control (MPC) [27, 20]. These methods are particularly effective for low-dimensional tasks, including balancing, set-point tracking, or waypoint following, where they can generate stable and accurate trajectories. However, they often struggle in complex or high-dimensional environments.
- **rule-based and planning-based systems:** For manipulation and navigation tasks, motion planning algorithms such as A*, PRM, or RRT* can be used to compute collision-free trajectories. Frameworks such as MoveIt [21] integrate these planning methods with low-level robot controllers and are widely used in robotics applications [14]. These approaches can generate high-quality demonstrations but often require accurate environment models and can become computationally expensive in complex scenarios.
- **policies obtained via reinforcement learning:** Another approach is to train a reinforcement learning policy, froze it and use it to generate expert demonstration trajectories for downstream imitation learning. Compared to hand-crafted or planning-based approaches, reinforcement learning policies can better adapt to high-dimensional and complex tasks, although training them typically requires large amounts of interaction data and computational resources.

In simulation, these policies can additionally be trained using *privileged simulation information*, which would not be accessible in real-world settings. Examples include exact object positions, velocities, segmentation masks, or the full environment state. Access to this additional information can simplify the learning process and improve the quality of the generated trajectories.

Another opportunity is to simplify the problem by training separate expert

policies for each subtask. For example, train one policy for grasping, one for navigation, and one for the pulling motion for opening drawers and coordinate them to form a complete task.

Online Learning in Simulation. In addition to generating demonstration datasets, simulation is widely used for online learning in robotics, particularly in reinforcement learning. In online learning, the robot continuously interacts with the environment and improves its policy based on newly collected experience during training. Simulation is especially suitable for this process because it enables safe exploration and automatic resetting of environments, parallelized data collection, and faster-than-real-time execution without risking damage to physical hardware. Consequently, simulation has become an essential component for scalable robot learning systems that require large amounts of interaction data, particularly in RL [336, 127, 190].

Sim-to-Real Gap. Despite its advantages, simulation-based robot learning faces several important challenges [25]. One major issue is the sim-to-real gap, which arises from inaccuracies between simulated and real-world physics. Effects such as friction, contact dynamics, deformable objects, latency, and sensor noise are often difficult to model accurately. This gap can lead to poor performance when transferring policies trained in simulation to real robots [329]. A common strategy to mitigate this problem is domain randomization, where environmental properties such as textures, lighting conditions, physical parameters, and sensor characteristics are randomized during training to improve robustness and transferability [266].

Task Diversity. Another limitation is the difficulty of creating sufficiently diverse simulated tasks and environments for general-purpose robot learning. Manually designing large numbers of environments is labor-intensive and does not scale efficiently. Recent work therefore explores procedural environment generation, where environments and task configurations are generated automatically to increase diversity and improve policy generalization [62, 262, 292, 76, 286].

Despite these challenges, simulated demonstrations remain a valuable source of training data for imitation learning, as they enable scalable trajectory generation and rapid experimentation in controlled environments.

- **Internet data.** Internet data is a promising source for robot learning and includes various types of freely available data, such as text, images, videos, and structured information. Its main advantage lies in its scale and diversity, which can improve the robustness, adaptability, and generalization capabilities of learned robot policies. In analogy to recent advances in natural language processing and computer vision, robotics can similarly benefit from leveraging vast, weakly labeled data from the web.

In particular, these data sources can support robot learning in several complementary ways. For example, image and video datasets have been used to pretrain vision models that provide robust visual representations for robotic perception

systems. In addition, Vision-Language Models (VLMs) and Large Language Models (LLMs) have been employed to define reward functions and task objectives for robot learning. LLMs have also been used as high-level planners for long-horizon tasks by decomposing complex goals into smaller executable actions. Furthermore, several recent works adapt internet-pretrained foundation models for robotics through fine-tuning on robot-specific datasets, while others jointly train on both internet data and action-labeled robot trajectories to improve generalization and cross-domain transfer capabilities.

In this work, the focus lies on internet video data, which is discussed in more detail in the following sections.

Combination of data sources. In modern robot learning systems, simulation data, real-world data, and internet data are rarely used in isolation. Instead, these data sources are commonly combined, as each category provides complementary strengths and addresses different limitations. Simulation enables scalable and cost-efficient data generation and provides a safe environment for online robot interaction and reinforcement learning [26, 36, 26, 23]. Internet data offers large-scale and diverse semantic information [9, 6], while real-world data captures accurate physical interactions, sensor characteristics, and real-world dynamics that are difficult to model in simulation [23].

Typical robot learning pipelines therefore combine these data sources hierarchically [6, 24]. In many approaches, perception and policy models are first pretrained on large-scale simulation and/or internet datasets in order to learn robust and generalizable representations. Subsequently, the models are fine-tuned on smaller but high-quality real-world robot datasets to adapt them to specific robotic platforms, sensors, and target environments [9]. Some works additionally employ hybrid training strategies that combine simulated and real-world data using techniques such as domain randomization or domain adaptation to further reduce the gap between simulated and real-world observations [32, 11].

4.2.4 Reduce the amount of robot data

To train a generalist robot, large amounts of diverse data are required in order to capture the variability of real-world environments and tasks. In particular, robot interaction data, also known as robot data, is highly valuable because it contains both observations of the environment, such from RGB-D cameras or other sensors, and the corresponding robot trajectories and actions. This enables the robot to learn the relationship between observations, actions, and their resulting outcomes.

However, the currently available amount of robot data is still insufficient in both scale and diversity to train highly generalizable robotic systems. Collecting robot data, whether in real-world environments or in simulation, is expensive and time-consuming. Real-world data collection requires physical hardware, human supervision, and extensive interaction time, while simulation environments often suffer from the sim-to-real gap and still require significant engineering effort. As a result, data scarcity remains one of the central challenges in the development of general-purpose robotic systems.

Large-scale internet video data. To adress this data bottleneck, one promising approach is to leverage large-scale internet video data. Internet videos provide a rich source of information about human behaviors, object interactions, and task execution in diverse environments that are relevant for robotics. They contain fundamental information about physical dynamics, a wide variety of scenarios and interactions (motion patterns), and semantic task structure that can be useful for robotic learning. In contrast to robot datasets, internet-scale video data is abundant, inexpensive to obtain, and covers a much broader range of scenarios and activities as robot datasets.

Although such videos typically do not contain direct robot actions, recent work has shown that they can still provide strong supervisory signals for representation learning, action understanding, and policy pretraining. By augmenting traditional robot datasets with large-scale video data, robotic systems can learn more robust and transferable representations, improving their ability to generalize to unseen tasks and environments [15].

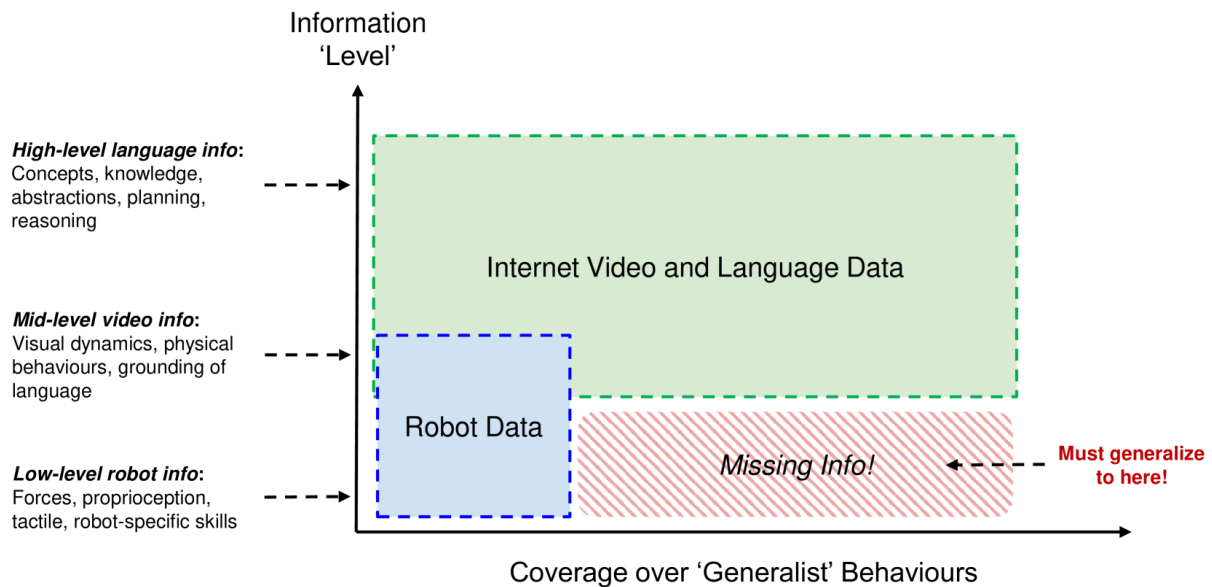


Figure 4.2: Problem: Data Bottleneck in Robotics. Source: [18].

Formalsim.

- D : The complete training dataset used for learning the robotic policy. This dataset is a combination of the categories real-world data, simulated data, and internet data. D can be subdivided into two subsets: $D = \{D_{robot}, D_{video}\}$.
- D_{robot} : Subset of dataset D containing robot interaction data collected from physical robots in real-world or simulation environments, including observations and the robots corresponding actions obtained during interaction. Each sample in D_{robot} consists of transition tuples of the form (o_t, a_t, r_t, o_{t+1}) , where o_t is the observation at time t , a_t is the action taken by the robot, r_t is the reward received after taking action a_t , and o_{t+1} is the resulting observation after executing action a_t . The reward r_t may be missing, and each o contains an image observation and can additionally contain other information, such as depth or tactile data.

- D_{video} : Subset of the dataset D that consists the video (demonstration) data. D_{video} can be obtained from external sources such as internet videos. A video clip is represented as $\tau = (o_0, o_1, \dots, o_T)$, where τ denotes the full video clip and each observation o_t corresponds to an RGB image. Depending on the dataset, videos may additionally be paired with language descriptions, action annotations, or other auxiliary modalities.

Figure 4.2 illustrates the data bottleneck in robotics. The x-axis represents the spectrum of behaviors that a generalist robot is expected to perform, while the y-axis reflects the different levels of information present in various data sources. The figure highlights that, although internet data encompasses a broader range of behaviors in diverse real-world scenarios compared to specialized robot datasets, it often lacks the detailed, low-level information that is critical for robotic applications. Ideally, for every relevant behavior, both low-level robot data and internet data would be available, allowing a model to establish connections across different information levels. However, as illustrated in Figure 4.2, there is a significant lack of low-level robot data for many behaviors that are well represented in internet data (see “Missing Info” area). This absence of fine-grained information poses a significant challenge for generalizing robot learning beyond the limitations of available robot datasets. Overcoming this gap and effectively leveraging both broad internet data and precise robot data is a central objective in advancing LfV approaches [18].

Goal of Learning from Video (LfV). LfV aims to exploit this abundance of video data to improve robotic learning. [18] says, the goal of LfV is to leverage our combined LfV dataset, $D = \{D_{robot}, D_{video}\}$, to obtain an improved policy $\pi(a_t|s_t)$ versus when learning from D_{robot} alone.

Improvements of LfV. What can we expect from combining both data sources, D_{robot} and D_{video} , instead of using only D_{robot} ?

Generalization beyond D_{robot} . One important improvement of Learning from Video (LfV) is the potential to generalize beyond the limited set of tasks contained in the robot dataset D_{robot} . While robot data primarily provides information about low-level robotic skills, such as grasping or locomotion, internet-scale video data contains a much broader variety of tasks and human behaviors. By combining both data sources, models may learn how low-level robot actions relate to higher-level task structures observed in videos. As a result, an LfV system may be capable of solving tasks that are not explicitly present in D_{robot} but are represented within D_{video} .

Emergent Capabilities. In natural language processing and computer vision, the availability of large-scale internet data has led to significant improvements and, in some cases, the emergence of capabilities that were not explicitly trained for. Similar progress is expected in robotics. By combining robot interaction data with internet-scale video data with language annotations, models may learn to connect low-level robotic skills with rich semantic abstractions and higher-level world knowledge obtained from textual data. This combination has the potential to improve generalization, task understanding, and the ability to solve previously unseen tasks.

Improvements in-distribution of D_{robot} . Furthermore, video data may also improve performance on tasks that are already represented within the robot dataset D_{robot} . Since robot demonstrations are expensive to collect, additional video data can improve data efficiency and help models learn more robust visual and task representations. As a result, Learning from Video (LfV) approaches may achieve higher task success rates and overall better performance compared to approaches trained solely on robot interaction data.

Information Levels: Robot data primarily contains low-level and mid-level information, and can be collected from both real-world and simulation, with real-world data being more valuable. In contrast, internet data is typically richer in mid-level and high-level information. The following sections explain each information level in detail.

- **Low-level robot information.** Low-level robot information refers to sensory and control signals that directly describe the robot's physical state, motion, and interaction with its environment. This includes, for instance, end-effector pose, joint positions, joint velocities, joint torques, and gripper state (e.g., open or closed). Additionally, low-level information includes information about the physical forces acting on the robot itself. In simulation environments, they may additionally include physical details of objects in the environment, such as contact forces, object properties or object dynamics. Such low-level information is essential for learning precise motor control tasks, including grasping, locomotion, and manipulation.

Proprioceptive information refers to the robot's internal sense of its own body and movement. It includes data about the robot's end-effector pose which implicitly includes joint angles, joint velocities, and gripper configuration. This information is crucial for executing accurate and stable motions, as it allows the robot to understand its current state and adjust its actions accordingly. A robot may visually observe a cup through a camera and wants to grasp it. Proprioceptive information tells the controller how to move the arm to the cup, which includes:

- what is the current pose/state of the robot
- how far the arm has moved,
- whether the wrist is rotated correctly,
- whether the gripper is aligned with the handle.

Tactile data refers to information obtained through physical contact between the robot and its environment. It is comparable to the human sense of touch. Common tactile signals include contact forces, pressure distributions, slip detection, surface texture information, and vibration signals. These measurements are typically acquired using sensors placed on robot grippers, fingertips, or other contact surfaces. For instance, during grasping, tactile sensors can indicate whether an object has been successfully grasped, whether it is slipping, or whether excessive force is being applied. This feedback is critical for maintaining stable and safe

object manipulation.

- **Mid-level video information.** Mid-level video information refers to visual observations that capture the dynamics of objects, agents, and environments over time without providing explicit low-level control signals. In contrast to robot data, video data does not include direct information about actions or internal robot states. Instead, it provides sequences of RGB observations that encode how the visual scene evolves over time.

This type of information includes visual dynamics such as object motion, changes in spatial configurations, and interactions between entities in the scene. For example, videos can capture how objects are moved, grasped, or manipulated by humans, as well as how these actions affect the surrounding environment. Although no explicit force or control signals are available, the physical behavior of objects (e.g., sliding, rotating, or deforming) can often be inferred implicitly from the visual changes across frames.

A key aspect of mid-level video information is that it encodes **physical behavior at a perceptual level**. While it does not provide direct measurements of forces or torques, it still reflects underlying physical principles such as gravity, friction, and object permanence through observable motion patterns. For instance, a video of a person pouring water into a glass implicitly contains information about liquid flow dynamics and object interaction constraints.

In addition, mid-level video information provides a natural source for **language grounding**. Many video datasets are paired with textual annotations, captions, or narration, which link visual observations to semantic concepts. This allows models to associate language with observed actions and events, such as "opening a door", "placing an object on a table", or "picking up a cup". Through such alignment, video data helps bridge the gap between perception and high-level task understanding, enabling learning of more abstract representations that go beyond purely visual dynamics.

- **High-level language information.** High-level language information refers to abstract semantic descriptions of tasks, concepts, and behaviors expressed in natural language. In contrast to visual or robot-centric signals, language does not encode direct sensory observations or motor commands, but instead provides structured knowledge about goals, constraints, and task structures. This includes descriptions of concepts (e.g., objects and their properties), relational knowledge, and symbolic abstractions of actions and environments.

Such information is particularly important for representing task intent and enabling generalization across different embodiments and environments. Language can describe tasks at multiple levels of abstraction, ranging from low-level action descriptions (e.g., "pick up the cup") to high-level procedural or goal-oriented instructions (e.g., "prepare a table for breakfast"). This hierarchical structure allows models to connect concrete behaviors with more abstract task specifications.

In addition, language plays a central role in **planning and reasoning**. It provides a

medium for expressing sequential structure, subgoals, and dependencies between actions. For example, a task such as “clean the table” can be decomposed into intermediate steps such as removing objects, wiping the surface, and reorganizing items. Such decompositions reflect implicit reasoning about task structure and enable more systematic policy learning.

Furthermore, language serves as a bridge for incorporating prior knowledge into robotic systems. Through textual descriptions, models can access commonsense knowledge about object affordances, physical interactions, and typical human behaviors. This enables improved generalization to unseen tasks by leveraging semantic relationships that are not explicitly present in robot or video data alone.

4.2.5 Challenges of Video Data

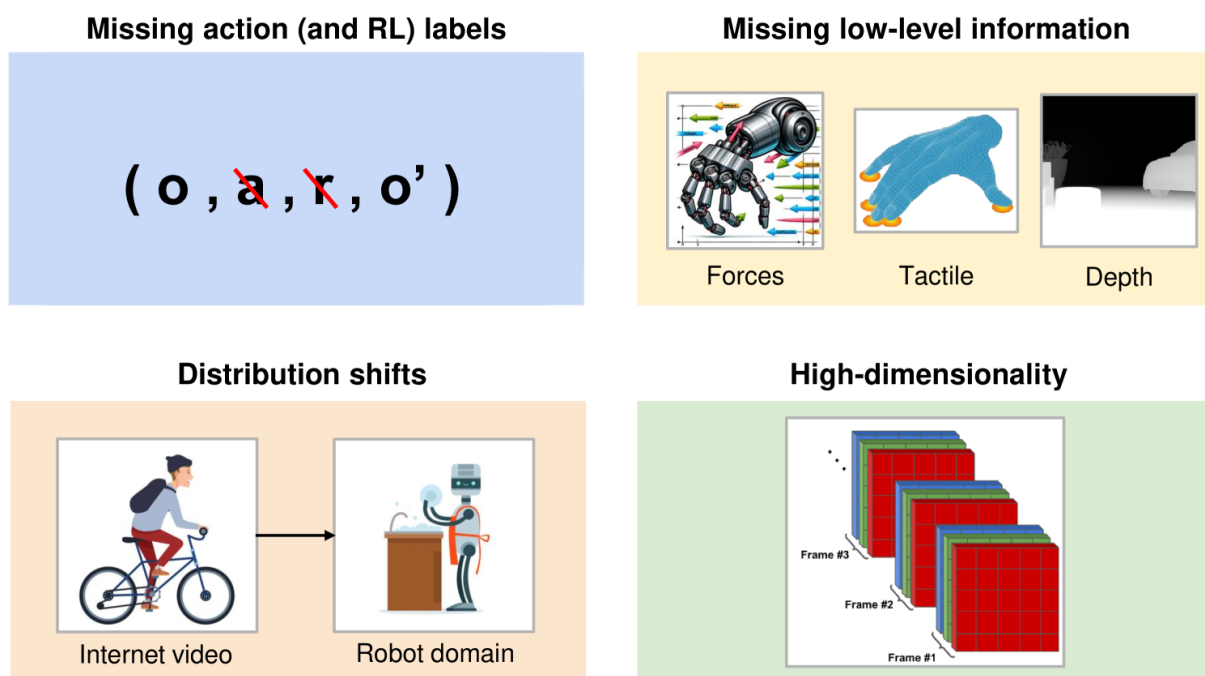


Figure 4.3: Key challenges in Learning from Videos [18].

Although internet-scale video data offers significant potential for robotic learning, it also introduces several fundamental challenges, see Figure 4.3. In contrast to robot interaction datasets, videos are typically high-dimensional, noisy, weakly labeled, highly diverse and lacks on low-level information. Extracting meaningful representations and task-relevant information from such data is therefore a non-trivial problem.

Missing Action Labels. One major limitation of raw video data is the absence of explicit action labels required by many imitation learning and offline reinforcement learning approaches. While videos provide rich visual observations of behaviors, they typically do not contain the low-level control commands executed during the task. Retrospectively inferring robot actions from human videos is challenging, since human motions often cannot be directly mapped to the action space of a robotic system.

Furthermore, videos usually lack additional reinforcement learning signals such as reward annotations, goal labels, or start-of- and end-of-episode boundaries.

Missing Low-level Information. Internet videos also lack on low-level sensory information. Skillful behaviors often depend on tactile feedback, force measurements, proprioceptive signals, or depth information, which are generally not observable in classical RGB video. Although videos may capture high-level task structure and object interactions, they do not explicitly provide the fine-grained physical information necessary for learning accurate motor control and stable interaction with the environment.

Distribution Shift. Another challenge arises from the distribution shift between internet videos and the downstream robotics domain. Differences may occur in camera viewpoints, environments, embodiments, object appearances, and task distributions. The Human demonstrations in videos may contain behaviors that are inefficient, ambiguous, or irrelevant for robotic execution.

In particular, Learning from Video (LfV) must address the embodiment gap between humans and robots. Human operators and robotic systems often differ significantly in their kinematics, morphology, and control capabilities. Furthermore, there exists a visual gap between the human actor performing the task in the video and the robotic embodiment that is expected to execute the task. As a result, transferring observed human motions directly to robot actions is highly non-trivial and requires models to learn embodiment-invariant task representations.

Additionally, some important robotic tasks, such as specialized industrial manipulation tasks, may be difficult to learn because they are underrepresented in publicly available video datasets.

High-dimensionality, Noise, and Redundancy. Video data is inherently high-dimensional and often contains substantial amounts of irrelevant or redundant information. Background changes, camera motion, occlusions, and unrelated actions can make it difficult for models to identify task-relevant features, making it challenging to learn robust representations. Furthermore, the sheer volume of video data can lead to computational challenges in processing and learning from such large datasets.

Controllability, Stochasticity, and Partial Observability. In unlabeled video data, it can be difficult for models to determine which changes in the environment are caused by the agent's actions and which originate from external factors or environmental noise. This ambiguity makes it challenging to infer action information directly from video observations. Furthermore, real-world environments are frequently stochastic and only partially observable, meaning that important state information may be hidden from the camera viewpoint. As a result, predicting future states or accurately modeling action consequences from video alone becomes a difficult problem.

None of these challenges have yet been fully solved. In the following sections, we will discuss some possible ways to address these issues coming with internet video data. Nevertheless it seems likely that scaling — with the help of internet video data — can provide significant gains in robotic systems within the near-future [18].

4.2.6 Other Paths to Generalist Robots.

Goal: reduce the amount of robot videos

- **Video provides foundational information:** - Contains physics and dynamics information - Shows human behaviors, tasks and actions relevant to robots - High quantity and diversity - Relatively untapped resource - —>can be highly informative for general-purpose robots. use of diverse internet data has facilitated a transition from task-specific models to monolithic foundation models with more general capabilities

LfV will solve the data bottleneck by augmenting traditional robot data with large-scale internet video.

Chapter 5

Learning from Video Techniques

Videos are a challenging data modality, as you have seen in Section 4.2.5. This section addresses the two key questions for LfV research:

1. How to extract relevant knowledge from internet video?
2. How to apply video-extracted knowledge to robotic systems for policy learning and control?

We first review different approaches for extracting foundational knowledge from internet video, including video encoders, video prediction models, and video-to-text models. We then discuss methods for transferring video-extracted knowledge to robotics, addressing challenges such as missing action labels and distribution shift. Finally, we explore the intersection of LfV and reinforcement learning, highlighting how video-based learning can enhance RL algorithms and enable more efficient policy learning from visual data.

5.1 Extracting Foundational Knowledge from Internet Video

Recent progress in **Video Foundation Models (Video FMs)** provides a promising direction for addressing the first answer: How to extract robotic relevant knowledge from internet video. Video foundation models are large-scale deep learning models trained on massive internet video datasets. Through large-scale training, these models learn general visual and temporal representations that capture semantics, motion dynamics, physical interactions, and behavioral patterns. As a result, they can extract foundational knowledge that is transferable across a wide range of downstream tasks, including robotics applications.

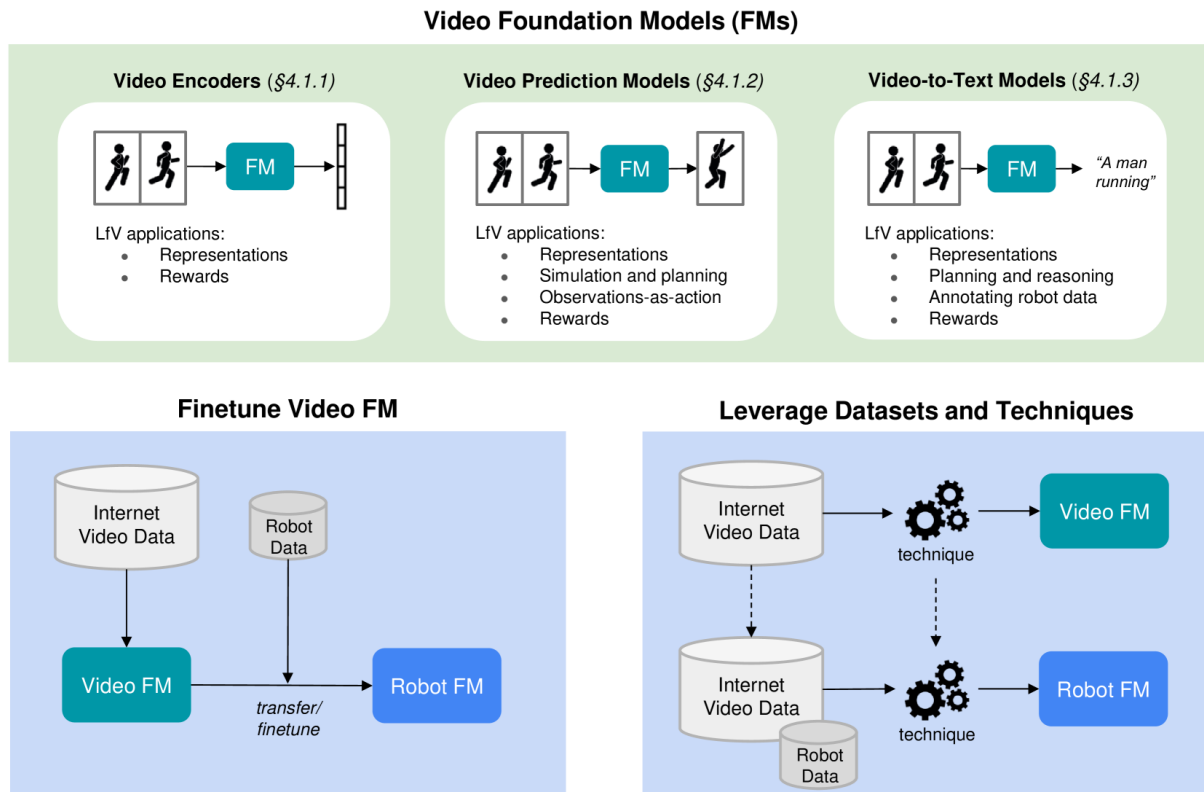


Figure 5.1: Video foundation modelling for Learning from Videos [18].

[18] says, Video FMs can be highly applicable for the following LfV use-cases, see Figure 5.1:

- **Using pretrained FMs:** One important application of Video FMs is the transfer of pretrained models to robotics tasks. For example, a video prediction model trained on internet video data may be adapted or fine-tuned into a robot dynamics model, allowing the robot to leverage the broad gathered knowledge about object motion and environmental dynamics from the pretrained Video FM.
- **Using techniques and datasets inspired by Video FMs:** In addition, techniques developed for video foundation models can be integrated into robot learning pipelines and can lead to better extraction of representations and knowledge. This, in turn, can lead to improved generalization and task understanding. For instance, a Robot FM can be directly trained by combining robot data with large-scale video inspired by video FMs techniques [251].

5.1.1 Video Encoders

Foundational video encoders $p(z \mid \tau)$ are capable of extracting rich and robust representations from raw video observations. Given a video clip τ , the encoder maps the visual input into a latent representation z , which captures semantic and temporal information relevant for downstream robotic tasks.

In robotic imitation learning, pretrained video encoders are commonly used in two ways:

- **Representation Learning:** Pretrained video encoders can be transferred to robotic learning frameworks and further fine-tuned on robot-specific data for downstream tasks such as policy learning or an RL Knowledge Modality. The learned latent representations provide compact and informative features that improve generalization and sample efficiency, by allowing the model to learn task-specific features while still leveraging the general knowledge captured from internet video.
- **Reward Learning:** Frozen video representations can also be used to define reward functions for robotic agents. By comparing the latent representations of demonstrations and robot executions, the system can measure behavioral similarity without requiring manually engineered reward signals. This approach enables the robot to learn from video demonstrations by optimizing its behavior to match the latent representations of successful demonstrations, thus facilitating imitation learning and skill acquisition.

5.1.1.1 Methods

General Overview of of Video Representation Learning.

Video representation learning aims to learn meaningful latent representations from raw video data. These representations capture semantic, spatial and temporal information that can be used for various downstream tasks.

How to learn a video encoder?

Let

- X denote a video clip
- $\bar{X}$ denote a video clip paired with some corresponding labels (bounding boxes, object masks) and data modalities (language annotations, meta-data)
- Y denote a learning objective, derived from $\bar{X}$

Learning goal during training: predict Y from X . Or in other words, learn a video encoder $p(z|X)$ that maps the video clip X to a latent representation z that can be used to predict Y : $X \xrightarrow{p(z|X)} z \rightarrow Y$.

Basic techniques for learning Y include:

- **Prediction-based Objectives:** The target Y , which is derived from the annotated video clip $\bar{X}$, may correspond to future video frames, masked regions, motion information, or other supervision signals. For example, in video prediction tasks, the model predicts the next frame X_{t+1} from the raw previous observations $X_{\leq t}$.
- **Joint-Embedding Objectives:** Joint-embedding approaches first map both the

video input X and the target Y into a shared latent embedding space:

$$z_X = f(X), \quad z_Y = g(Y).$$

A loss function is then applied to align semantically similar embeddings.

$$\mathcal{L}_{\text{pred}} = L_{\text{align}}(z_X, z_Y)$$

Contrastive learning is one of the most widely used approaches in this setting.

State-of-the-Art Approaches

Recent work has proposed scalable methods for learning rich spatio-temporal video representations.

Video-text objectives leverage language annotations to learn semantic representations from video data. Common approaches include:

- **Video-Text Contrastive Learning:** This approach learns to align video representations with corresponding text descriptions. By maximizing the similarity between video and text embeddings, the model captures semantic relationships between visual content and language.
- **Video-Text Matching:** This method trains the model to determine whether a given video clip and text description correspond to each other. The model learns to distinguish between matching and non-matching pairs, which encourages it to capture relevant features for both modalities.
- **Masked Language Modelling:** In this approach, the model learns to predict masked words in a text description based on the corresponding video content. This encourages the model to learn contextual relationships between visual information and language.
- **Video-to-Text Generation:** This method trains the model to generate descriptive text based on video input. By learning to produce coherent and relevant descriptions, the model captures high-level semantic information from the video data.

Auto-encoding approaches have also become highly effective for self-supervised video representation learning. Auto-Encoding Approaches learn video representations by reconstructing input data from compressed latent representations. Common approaches include:

- **Masked Auto-Encoding (MAE):** In this approach, parts of the video input are randomly masked, and the model is trained to reconstruct the missing regions from the visible observations. This encourages the encoder to learn meaningful spatial and temporal representations from incomplete video information.
- **Vector-Quantized Auto-Encoding (VQ-AE):** In this approach, the encoder first extracts continuous latent features from the video input. These features are

then quantized by mapping them to the nearest vector in a finite set of learned codebook embeddings/vectors. The decoder reconstructs the original video from these vector, encouraging the model to learn compact and semantically meaningful representations, due to the limited set of learned embeddings. The underlying principle is similar to language modelling, where a finite vocabulary of discrete tokens is used to represent words, syllables or letters. Analogously, VQ-AE learns a finite set of discrete visual tokens that capture recurring patterns in video data. These tokens may represent object appearances, motion patterns, textures, actions, or scene structures.

Student-Teacher Distillation Approaches learn representations by transferring knowledge from a pretrained teacher network to a smaller student model. The student is trained to match the representations or predictions generated by the teacher model. This encourages the student network to learn robust and generalized video representations while often requiring fewer computational resources.

Joint-Embedding Predictive Architectures (JEPA) learn representations by predicting high-level latent embeddings instead of reconstructing raw video pixels. Rather than modeling every visual detail, the model predicts semantic representations of future or masked observations in a shared embedding space. This approach improves robustness against the high-dimensionality and noise commonly present in video data while focusing on semantically relevant information.

5.1.2 Video Prediction Models

Video prediction models (VPMs) aim to model the conditional distribution over future visual observations given past context. The standard formulation is next-frame prediction

$$p(o_{t+1} \mid o_{t-k:t}),$$

while more generally VPMs can be expressed as conditional video generation models

$$p(\tau \mid c),$$

where τ is a video sequence and c denotes conditioning information such as past observations, actions, robot actions, language instructions, or goal images. Trained on large-scale video data, these models implicitly learn priors over physical dynamics, object interactions, and human behavior, making them relevant for robotic learning and imitation learning.

During training, the model learns to predict future visual outcomes from past context, either in pixel space or in a learned latent space:

Relevance for Robotics.

Video prediction models are useful in robotics in several complementary ways:

- **Dynamics Models:** Video prediction models can be extended into action-conditioned dynamics models of the form

$$p(o_{t+1} \mid o_{t-k:t}, a_t),$$

to serve as simulator or planner. In this setting, the model predicts future observations conditioned on actions, which allows model-predictive control by evaluating candidate action sequences in visual or latent space.

- **Policy Learning:** VPMs implicitly learn the distribution of behaviors present in the training data by modeling how visual scenes evolve over time. This allows them to generate multiple plausible future trajectories given a current observation. In a robotics context, these generated trajectories can be interpreted as candidate action outcomes.

This enables an “observation-as-action” perspective, where control is not performed by directly predicting actions, but by evaluating or selecting among predicted future video rollouts. The resulting policy can be viewed as choosing the trajectory that best aligns with a desired objective (e.g., reaching a goal state or matching expert demonstrations), thereby directly connecting video prediction with imitation learning and planning.

- **Representation Learning:** Learned latent spaces capture semantically meaningful information about objects, motion, and interactions, which can be transferred to downstream imitation learning and reinforcement learning tasks.
- **Reward Learning:** Video prediction models can also be used to define reward functions by measuring how well a robot’s predicted future aligns with desired behaviors. For example, a reward signal can be derived from the similarity between generated trajectories and expert demonstrations in latent space, or from the likelihood of a trajectory under a learned video prior. This enables imitation learning without manually engineered reward functions.

5.1.2.1 Methods

Modern video prediction approaches are dominated by diffusion models, autoregressive transformers, and masked transformers. Each of these approaches has unique strengths and limitations, and they can be combined with various conditioning mechanisms to enable controllable video generation for robotics applications.

Diffusion Models.

Diffusion-based video prediction models generate future frames through an iterative denoising process. During generation, the model starts from random sampled noise and progressively reconstructs a plausible future video sequence conditioned on previous observations, actions, or other conditioning inputs. The denoising model learns to reverse a forward corruption process in which noise is gradually added to training samples.

Diffusion models are particularly effective at modeling complex continuous distributions and producing high-fidelity video samples. However, they suffer from two main limitations: slow sampling speed due to the iterative denoising process, and difficulty maintaining long-horizon temporal consistency across generated frames.

To improve computational efficiency, many approaches perform diffusion in a learned latent space rather than directly in pixel space. In addition, many methods leverage large-scale image datasets alongside video data during training to improve visual quality and generalization. Below are the key differences between pixel-space and latent-space diffusion models:

- **Pixel-Space Diffusion:** In pixel-space diffusion, noise is added directly to raw video frames during training. The model learns to denoise these noisy frames to reconstruct the original video conditioned on context information such as previous frames, actions, or language instructions. While this approach can capture fine-grained visual details, it is computationally expensive and may struggle with long-horizon consistency due to the high-dimensionality of pixel space.
- **Latent-Space Diffusion:** In latent diffusion models, the video frames are first encoded into a lower-dimensional latent representation using an encoder before noise is added. The diffusion process is then performed in this compressed latent space instead of directly on pixels. The denoised latent representation is subsequently decoded back into video frames using a decoder. Operating in latent space reduces computational cost and removes low-level pixel redundancy while preserving the high-level semantic and temporal structure of the video, leading to improved efficiency and better long-horizon consistency.

Autoregressive and Masked Transformer Models.

Rather than generating future frames through iterative denoising like diffusion models, transformer-based approaches model videos as a sequence of discrete tokens, typically obtained from a vector-quantized autoencoder (VQ-AE). The encoder maps the input video into a lower-dimensional continuous latent representation. These latent features are then quantized by replacing each feature vector with the nearest embedding vector from a learned codebook. As a result, the video can be represented as a sequence of discrete visual tokens, analogous to words in natural language processing.

Autoregressive transformers generate future tokens sequentially by predicting the next token conditioned on previously generated tokens. While effective at modeling temporal dependencies, small prediction errors can accumulate over long horizons, leading to temporal drift and reduced generation quality.

Masked transformer models address this limitation by masking subsets of video tokens and predicting multiple missing tokens in parallel instead of generating them strictly one-by-one. The model iteratively refines the masked predictions until a complete video sequence is reconstructed. During inference, the model can be conditioned on a short history of recent camera observations from the robot environment, which are

encoded into visible tokens, while the tokens corresponding to future time steps are initially masked. The model then predicts the missing future tokens for the entire horizon in parallel and iteratively refines these predictions until a coherent future video sequence is obtained. The model can also be conditioned on additional inputs such as language instructions, goal images, or robot state information, allowing it to generate future video predictions that are aligned with a desired task or behavior. Compared to autoregressive decoding, masked transformers improve computational efficiency and reduce long-horizon error accumulation.

Conditioning mechanisms

VPMs can be conditioned on actions, language, or goal images, enabling structured control over generated futures. In robotics, such conditioning effectively turns video prediction models into controllable world models, bridging perception, planning, and imitation learning. With such a world model, a robot can perform counterfactual reasoning over actions, simulate future outcomes of different control strategies, and evaluate how well predicted trajectories align with expert demonstrations or task objectives.

5.1.3 Video-to-Text Models.

Video-to-text models are multimodal systems that map video inputs to natural language outputs:

$$p(\text{text} \mid \tau),$$

where τ is a video sequence and *text* is a natural language description. These models are trained on large-scale video-text datasets and learn to extract semantic information from videos while generating coherent language outputs. Video-to-text models can be used to generate natural language descriptions of video content, for video question answering and for video summarization.

Relevance for Robotics.

Their usefulness in robotics can be summarized in the following aspects:

- **High-quality representations:** By learning to generate natural language descriptions of video content, video-to-text models learn to extract high-level semantic representations that capture object properties, actions, interactions, and scene context. These representations can be transferred to downstream robotic tasks, improving generalization and task understanding compared to video-only or image-to-text models. The learned representations can capture both high-level semantic information and temporal dynamics, which are critical for understanding complex robotic tasks and environments.
- **Grounded reasoning and planning:** Large language models (LLMs) have demonstrated strong capabilities as high-level planners in robotics. However, their lack of direct grounding in physical perception limits their reliability in real-world environments.

Video-to-text models address this limitation by conditioning reasoning directly on rich visual-temporal input. This enables:

- improved grounding of symbolic reasoning in observed physical states
 - better modeling of object interactions and environment dynamics
 - more consistent closed-loop reasoning, where predictions are continuously informed by perceptual feedback
- **Annotating robot data:** Video-to-text models can be used to generate high-quality language annotations for robot datasets. By automatically describing video demonstrations of robotic tasks, these models can create rich textual annotations that capture task-relevant information such as object properties, actions, goals or step-by-step task descriptions.
 - **Rewards and Feedback:** Video-to-text models can be used to provide reward signals or value estimates for reinforcement learning and imitation learning systems. By framing reward learning as a visual-question answering problem, the model can evaluate a robot’s behavior directly from video observations. For example, the model may answer questions such as “Did the robot successfully grasp the object?” or “Did the trajectory achieve the desired goal?”. This formulation is typically goal-oriented and explicitly tied to predefined task semantics. The predicted responses can then be transformed into reward signals that guide policy optimization.

In addition, video-to-text models can be integrated into reinforcement learning from AI feedback (RLAIF) frameworks, where the model evaluates the quality of predicted or executed behaviors relative to task objectives or expert demonstrations. The model compares different behaviors—either whole trajectories or segments of trajectories—and provides judgments such as which behavior is more successful, safe, or aligned with the desired objective. These ranking-based signals are then distilled into a reward model or value function.

5.1.3.1 Methods

Video-to-Text Methods Existing approaches can be broadly categorized into compositional pipelines, LLM-based architectures with adaptors, and natively multi-modal sequence models.

- **Compositional Approaches:** decompose the video-to-text problem into smaller sub-tasks. Typically, an Image-Language Model is first used to process individual video frames by answering frame-level questions. The resulting intermediate descriptions are then aggregated using a Large Language Model (LLM), which generates a global video summary. While effective in practice, these approaches do not involve end-to-end video training, and therefore often fail to learn rich temporal video representations.
- **Leveraging Pretrained LLMs via Adaptors and Finetuning:** A common strategy is to combine a pretrained video encoder with a pretrained LLM. Since LLMs

operate on token sequences, an adaptor module is introduced to project video features into the LLM input space. With the adaptor, the video encoder and LLM can be trained end-to-end on video-text data, allowing the model to learn to extract video representations that are aligned with language generation.

- **Natively Multi-Modal Models:** More recent approaches integrate video and text into a unified autoregressive framework, instead of combining separate models.

In natively multi-modal sequence models, video-to-text generation is formulated as an interleaved sequence modeling problem over visual and textual tokens. Video inputs are first tokenized into discrete visual tokens, which are then combined with text tokens into a unified sequence.

The model is trained autoregressively using next-token prediction over this joint sequence, i.e.,

$$p(x_t | x_{<t}),$$

where tokens may correspond to either visual or textual elements. This formulation enables a unified treatment of different modalities within a single transformer architecture.

As a result, such any-to-text or any-to-any models allow video, text, and potentially other modalities to interact within the same latent space. This leads to a tighter coupling between vision and language representations and moves towards more general multi-modal foundation models capable of flexible cross-modal generation and reasoning.

5.1.4 Challenges and Future Directions

Despite recent progress, video representation learning still faces several limitations:

- Current models may **hallucinate** or fail to capture fine-grained spatial relationships, precise spatio-temporal dynamics, and long-term temporal dependencies.
- Another key bottleneck is the **limited quality and scale of low-level annotated video data**, particularly for tasks requiring detailed fine-grained understanding.
- Additionally, processing **high-dimensional and long video sequences** remains computationally expensive, motivating more efficient model architectures.

Further directions include reinforcement learning-based finetuning to reduce hallucinations, the integration of 3D information, like depth to improve physical realism, and improved single-step conditioning mechanisms for fine-grained video prediction tasks.

5.2 Video-to-Robot Transfer: Addressing Missing Action Labels and Distribution Shift

The main challenges for transferring video-extracted knowledge to robotics are:

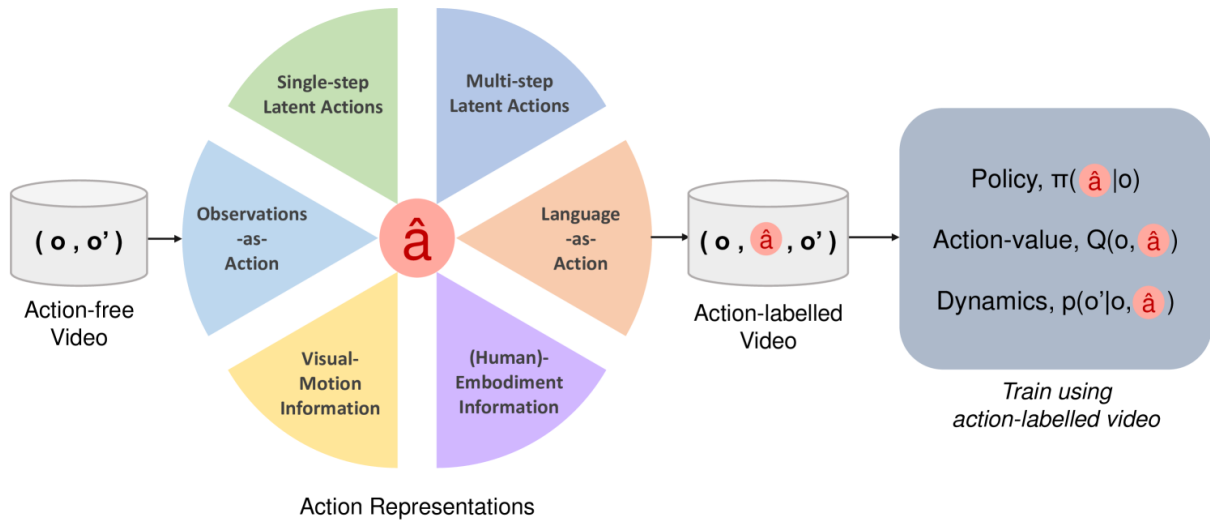


Figure 5.2: To label action-free videos, an Alternative Action $\hat{a}$ can be derived from video data instead of the missing action a [18].

- (i) **Missing action labels:** Most video datasets do not contain explicit robot action annotations, which are required for learning control policies and dynamics models.
- (ii) **Distribution shift:** There exists a significant domain gap between human or internet videos and robot execution. This includes differences in visual appearance, embodiment, dynamics, and behavior patterns, which can severely degrade the generalization of models trained on video data when applied to robotic tasks.

In many real-world video datasets, explicit robot action labels (e.g., joint velocities or torques) are not available. Raw video data provides only state transitions pairs (o_t, o_{t+1}) , without access to the corresponding actions a_t . This makes it difficult to directly apply Behavior Cloning or reinforcement learning methods, which typically require triplets of the form (o_t, a_t, o_{t+1}) . Recovering such low-level actions directly from video is a challenging problem due to ambiguity and the lack of physical interaction information. To overcome this limitation, recent approaches introduce an Alternative Action Representation $\hat{a} \in \hat{A}$, which can be used to relabel action-free videos and enable video-to-robot transfer despite missing action labels and distribution shift, see Figure 5.2.

Formally, this induces a relabeling of video transitions as:

$$(o_t, o_{t+1}) \rightarrow (o_t, \hat{a}_t, o_{t+1}), \quad \hat{a}_t \in \hat{A}.$$

What is an Alternative Action Representation?

Formally, an alternative action representation is defined as

$$\hat{a} \in \hat{A},$$

where $\hat{A}$ denotes an Alternative (latent or abstract) Action Representation Space derived from visual or multimodal video data. Instead of modeling actions as raw robot control signals, actions are encoded in a semantic feature space that captures high-level behavior such as grasping, moving, or pushing. In other words, $\hat{A}$ captures what actions *mean*, rather than how it is executed at the motor level. The key idea is that $\hat{a}$ serves as a proxy for the missing action labels, allowing the model to learn from video data without requiring explicit action annotations. Later, the $\hat{a}$ can be decoded or translated into the robot's actual action space if a mapping $f : A \rightarrow \hat{A}$ exist. Otherwise, they still provide valuable structure [278, 288, 67]. However it is also valid to define a $\hat{A}$ that requires labelled video (e.g., language captions).

Example: Consider a video of a human performing a pick-and-place task. The raw video data provides visual observations of the human's hand and the objects being manipulated, but does not include explicit robot action labels such as joint angles or end-effector velocities.

- The **Video Encoder** extracts visual features (what is being done).
- The **Action Embedding Space** $\hat{A}$ captures *abstract actions* such as *reach*, *grasp*, *move up*.
- The **Policy** $\pi_{alt}(\hat{a}_t | o_t, c)$, $\hat{a}_t \in \hat{A}$ predicts the next action *in this latent action space* based on current visual observations and probably some additional context information c .
- Later, a **Decoder** $\pi(a_t | \hat{a}_t, o_t)$ or **Alignment Model** $a_t = f_{align}(\hat{a}_t)$ maps these latent actions to the robot's real motor commands.

Why is $\hat{A}$ useful?

Working in the alternative action space $\hat{A}$ offers several advantages.

- **Bridging the domain gap:** $\hat{A}$ bridges the gap between human video data and robot control by focusing on action semantics rather than platform-specific motor commands.
- **Learning from weak labels:** $\hat{A}$ enables learning from raw or weakly labeled videos, since inferring high-level actions from video is easier than recovering low-level motor commands.
- **Transferable Representations across embodiments:** $\hat{A}$ supports transfer across embodiments, because the same semantic action (e.g., "open drawer") can be instantiated by different robots with different kinematics.
- **Structured learning:** A well-structured representation $\hat{A}$ that is minimal, disentangled, and semantically consistent across the state space encodes meaningful action information. This makes it easier to learn models that are predictive of both future observations (i.e., learning $p_{alt}(o_{t+1} | \hat{a}_t, o_t)$ instead of $p(o_{t+1} | o_t)$) and robot actions (i.e., learning $\pi(a_t | \hat{a}_t, o_t)$ instead of $\pi(a_t | o_t)$), leading to more

data-efficient and generalizable policies, dynamics models, and value functions. The choice of abstraction level for $\hat{A}$ also plays a crucial role:

- **Higher-level or longer-horizon representations** $\hat{A}$ (such as language-based actions) can benefit performance in complex, long-horizon tasks by providing broader context and intent. This is because they capture more abstract and semantically meaningful information that can guide decision-making over longer time scales, which is particularly beneficial for tasks that require planning and reasoning about future consequences [18].
- **Lower-level or shorter-horizon representations** $\hat{A}$, which retain more fine-grained information, may be more effective for accurately reconstructing or decoding the original robot actions a_t from the alternative actions $\hat{a}_t$ [18].

Once $\hat{A}$ is defined, the same representation can be used for example for:

- policies $\pi_{\hat{A}}(\hat{a}_t \mid o_t)$ [227],
- dynamics models $p(o_{t+1} \mid o_t, \hat{a}_t)$ [40] or
- state-action value functions $Q(o_t, \hat{a}_t)$ [27],

which further encourages consistent and transferable action abstractions.

5.2.1 Categories of Alternative Action Representations

There exist different approaches to derive alternative action representations $\hat{a} \in \hat{A}$ from video data. These methods differ in how actions are represented and which information is extracted from visual observations.

Single-step Latent Actions:

These approaches learn latent actions that describe the transition from one observation to the next:

$$\hat{a}_t : o_t \rightarrow o_{t+1}.$$

Methods:

- Forward Dynamics Models (FDM) $p(o_{t+1} \mid \hat{a}_t, o_t)$, which predict the future observation from the current observation and latent action.
- Inverse Dynamics Models (IDM) $p^{-1}(\hat{a}_t \mid o_t, o_{t+1})$, which infer the latent action responsible for a transition.

Limitations:

- Often capture all visual changes, not only agent actions.
- May omit important non-visual information such as forces or contact dynamics.

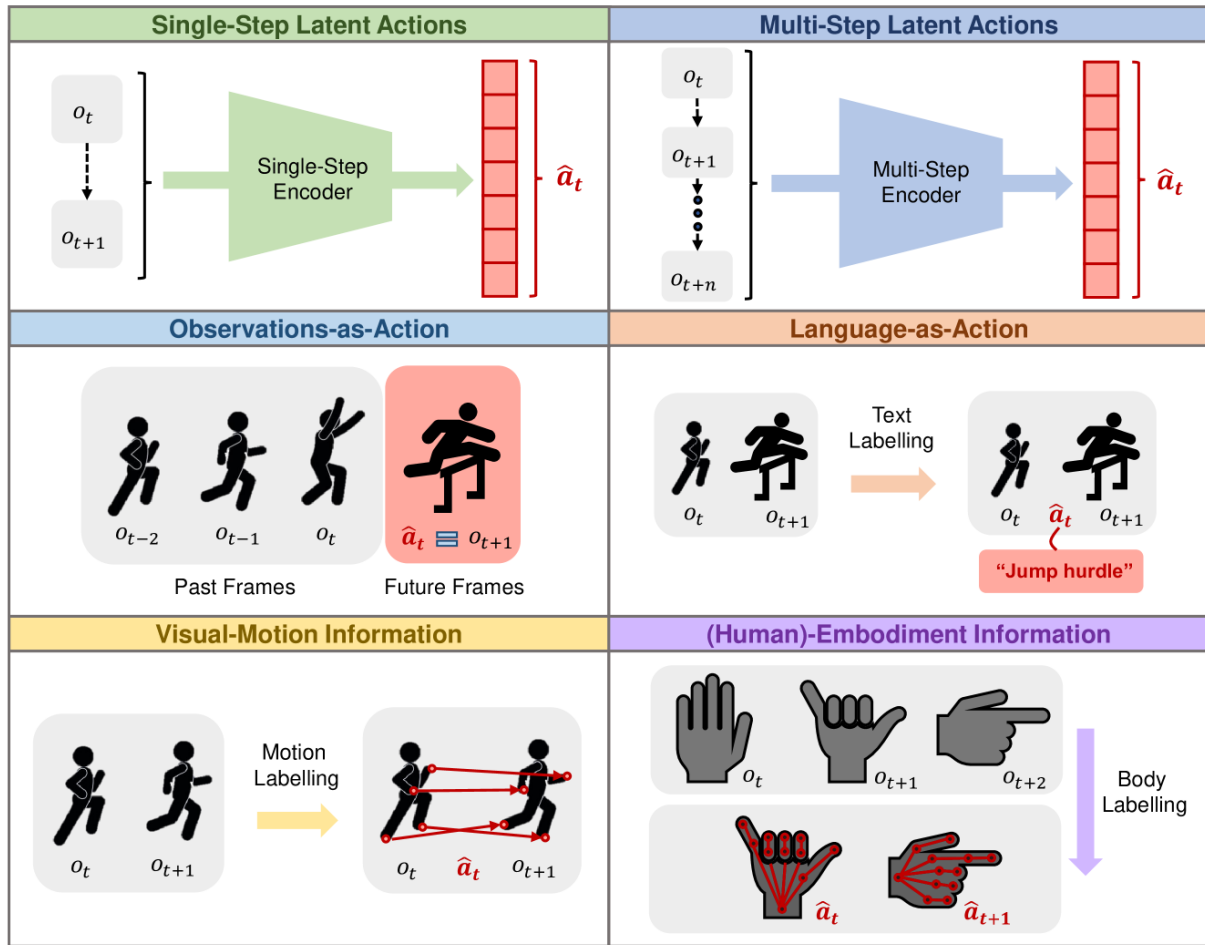


Figure 5.3: Categories of Alternative Action Representations that can be learned or obtained from video data [18].

Multi-step Latent Actions:

These methods encode behaviors across multiple time-steps:

$$\hat{a}_t : o_t \rightarrow o_{t+k}.$$

Instead of modeling individual actions, they capture higher-level behavioral patterns or latent plans. The key idea is to learn a compact latent representation of a *skill* or *sub-policy* that summarizes an entire sequence of actions. These representations are more abstract, goal orientated and can encode longer-horizon information, which may be beneficial for complex tasks.

For example these “skill embeddings” can capture structures such as “reach the object”, “move the arm to the target location”, or “grasp and lift”, which can later be decoded into a full action sequence.

Methods:

- Variational sequence models: These methods use variational autoencoders (VAE) to encode motion trajectories or sequences of observations into a latent variable

z that captures the underlying behavior. The VAE is trained to reconstruct the original trajectory from the latent representation.

- Video-language contrastive learning aligns video segments with natural language descriptions of behaviors.
- Supervised contrastive learning on action-labeled videos pulls videos with the same or similar action labels together in latent space, while pushing videos with different action labels apart.
- Self-supervised clustering approaches group similar motion patterns or trajectories into clusters without requiring labels, with each cluster representing a reusable skill or behavior primitive.
- Further approaches in [268], Pertsch et al.[204], and Cai et al. [42].

Limitations:

- Often produce abstract and long-horizon representations that may lose fine-grained control details and be harder to align with low-level robot actions.

Observations-as-Actions:

These approaches directly use (encoded) future observations as action representations:

$$\hat{a}_t = o_{t+k}.$$

Future observations provide information about the intended behavior or goal state.

Methods:

- Next-observation-as-action uses the next observation o_{t+1} as the $\hat{a}_t$ label and provides information regarding the action that should be taken.
- Observations-as-subgoals uses future observations as intermediate goals to guide action selection.
 - Fixed time horizon uses observations that are fixed k time steps in advance.
 - Key-frame selection identify critical states in a trajectory that can serve as action representations, see table 5.1.

Zeitpunkt	Frame-Beschreibung	Bedeutung
t_0	Arm über Tisch	Start
t_1	Arm nähert sich Tasse	Zwischenzustand
t_2	Hand berührt Tasse	✓ Key-frame
t_3	Tasse wird angehoben	Zwischenzustand
t_4	Tasse über Ablage	✓ Key-frame
t_5	Tasse wird abgestellt	Ende

Table 5.1: Beispielhafte Key-Frames in einer Pick-and-Place-Sequenz

Limitations:

- Future observations may not uniquely determine the underlying action.
- High-dimensional visual observations can increase learning complexity.

Language-as-Actions:

Natural language descriptions can be used as high-level action representations for example by captioning video frames with language instructions that describe the observed behavior:

$$\hat{a}_t = \text{"pick up the cube"}.$$

Language-based actions provide semantic supervision and enable integration with large language models, which can facilitate more complex reasoning and planning.

Methods:

- Language-captioned video datasets
- Automated or semi-automated video captioning
- Video-to-Text Models
- Vision Language Models (VLMs) and variants of them

Limitations:

- Language is often coarse and omits low-level control details.
- Requires annotated or captioned video data.

Visual Motion Information:

These approaches define actions using visual motion cues extracted from video sequences. The underlying assumption is that temporal changes in visual observations (motion patterns) contain implicit information about the actions responsible for observed state transitions. For example, the direction, magnitude, and trajectory of movement can provide information about the intended behavior, goal, or interaction dynamics.

Methods:

- 2D or 3D point trajectories of tracked objects or hands via off-the-shelf point trackers
- Optical flow prediction, which captures how image regions move between consecutive video frames
- Visual action arrows that represent movement direction and magnitude directly in the image

- Structure-from-motion approaches, which reconstruct 3D scene geometry and camera motion from video in order to recover information about underlying physical interactions

Limitations:

- Motion estimates can be noisy or ambiguous.
- Visual motion does not always correspond directly to robot actions.

(Human)-Embodiment Information:

These approaches explicit models human body structure and kinematics. In contrast to generic motion-based methods, which focus on pixel or object movement, these approaches aim to represent the *agent performing the action* using structured human pose information. Embodiment-specific information such as hand poses, affordances, or trajectories can provide useful cues about the intended actions and interactions in the video, which can be transferred to robotic embodiments. For example, an alternative action representation can be defined as:

$$\hat{a}_t = \text{hand pose at time } t + 1,$$

which encodes the intended end-effector movement and can be associated with actions such as reaching, grasping, or manipulating objects.

Methods:

- Human pose estimation (full-body kinematics)
- Hand pose estimation and hand trajectory tracking
- Affordance prediction (e.g., graspable regions, interaction points)
- Object- and hand-mask conditioned representations, which focus on the regions of the video that are relevant for action understanding and ignore irrelevant background information

Limitations:

- Human embodiment is structurally different from robotic embodiments, limiting direct transferability.
- Accurate pose and hand tracking or estimation can be unreliable in cluttered or occluded scenes.
- Focuses on human-centric actions, which may not generalize to non-human interaction styles.

Inverse Dynamics Model (IDM) pseudo-action Labels:

These methods train an inverse dynamics model on robot data and use it to infer pseudo-action labels for action-free (human) videos, which can then be used for

learning policies:

$$p^{-1}(a_t \mid o_t, o_{t+1}).$$

Limitations:

- Require action-labeled robot data for model training.
- Small domain shift between demonstration videos and robot execution.

5.3 LfV and Reinforcement Learning

Bibliography

- [1] OpenAI Josh Achiam et al. "GPT-4 Technical Report". In: 2023. URL: <https://api.semanticscholar.org/CorpusID:257532815>.
- [2] Yongjie Bai et al. *Learning to See and Act: Task-Aware View Planning for Robotic Manipulation*. Version 3. Oct. 28, 2025. DOI: 10.48550/arXiv.2508.05186. arXiv: 2508.05186 [cs]. URL: <http://arxiv.org/abs/2508.05186> (visited on 05/02/2026). Pre-published.
- [3] James Betker et al. "Improving Image Generation with Better Captions". In: ().
- [4] *Boosting Robotic Manipulation Generalization with Minimal Costly Data*. URL: <https://arxiv.org/html/2503.19516v2> (visited on 05/02/2026).
- [5] Anthony Brohan et al. "RT-1: Robotics Transformer for Real-World Control at Scale". In: 2022. arXiv: 2212.06817.
- [6] Anthony Brohan et al. "RT-2: Vision-language-action Models Transfer Web Knowledge to Robotic Control". In: 2023. arXiv: 2307.15818.
- [7] Tim Brooks et al. "Video Generation Models as World Simulators". In: (2024). URL: <https://openai.com/research/video-generation-models-as-world-simulators>.
- [8] Yiteng Chen et al. *RoboRouter: Training-Free Policy Routing for Robotic Manipulation*. Mar. 12, 2026. DOI: 10.48550/arXiv.2603.07892. arXiv: 2603.07892 [cs]. URL: <http://arxiv.org/abs/2603.07892> (visited on 05/02/2026). Pre-published.
- [9] Open X-Embodiment Collaboration et al. *Open X-Embodiment: Robotic Learning Datasets and RT-X Models*. 2023. URL: <https://arxiv.org/abs/2310.08864>.
- [10] Rohan Jagtap. *Understanding the Markov Decision Process (MDP)*. Built In. URL: <https://builtin.com/machine-learning/markov-decision-process> (visited on 04/28/2026).
- [11] Stephen James et al. "Sim-to-Real via Sim-to-Sim: Data-efficient Robotic Grasping via Randomized-to-Canonical Adaptation Networks". In: *CoRR* abs/1812.07252 (2018). arXiv: 1812.07252. URL: <http://arxiv.org/abs/1812.07252>.
- [12] Eric Jang et al. *BC-Z: Zero-Shot Task Generalization with Robotic Imitation Learning*. Feb. 4, 2022. DOI: 10.48550/arXiv.2202.02005. arXiv: 2202.02005 [cs]. URL: <http://arxiv.org/abs/2202.02005> (visited on 05/02/2026). Pre-published.
- [13] Jared Kaplan et al. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361 [cs.LG]. URL: <https://arxiv.org/abs/2001.08361>.

- [14] Zachary Kingston and Lydia E. Kavraki. “Robowflex: Robot Motion Planning with MoveIt Made Easy”. In: *CoRR* abs/2103.12826 (2021). arXiv: 2103.12826. URL: <https://arxiv.org/abs/2103.12826>.
- [15] Fanqi Lin et al. “Data Scaling Laws in Imitation Learning for Robotic Manipulation”. 2024. arXiv: 2410.18647.
- [16] Zhao Mandi et al. *CACTI: A Framework for Scalable Multi-Task Multi-Scene Visual Imitation Learning*. Feb. 16, 2023. DOI: 10.48550/arXiv.2212.05711. arXiv: 2212.05711 [cs]. URL: <http://arxiv.org/abs/2212.05711> (visited on 05/02/2026). Pre-published.
- [17] Robert McCarthy et al. “Towards Generalist Robot Learning from Internet Video: A Survey”. In: *Journal of Artificial Intelligence Research* 83 (July 21, 2025). ISSN: 1076-9757. DOI: 10.1613/jair.1.17400. arXiv: 2404.19664 [cs]. URL: <http://arxiv.org/abs/2404.19664> (visited on 05/04/2026).
- [18] Robert McCarthy et al. “Towards Generalist Robot Learning from Internet Video: A Survey”. In: *Journal of Artificial Intelligence Research* 83 (July 2025). ISSN: 1076-9757. DOI: 10.1613/jair.1.17400. URL: <http://dx.doi.org/10.1613/jair.1.17400>.
- [19] Tim Miller. *Markov Decision Processes — Mastering Reinforcement Learning*. URL: <https://gibberblot.github.io/rl-notes/single-agent/MDPs.html> (visited on 04/28/2026).
- [20] *Model Predictive Control Introduction and Setup*. Robotics Knowledgebase. May 4, 2022. URL: <https://roboticsknowledgebase.com/wiki/actuation/model-predictive-control/> (visited on 05/08/2026).
- [21] *MoveIt 2 Documentation — MoveIt Documentation: Rolling Documentation*. URL: <https://moveit.picknik.ai/main/index.html#> (visited on 05/08/2026).
- [22] *Multitask Robot Manipulation Policies*. URL: <https://www.emergentmind.com/topics/multitask-robot-manipulation-policies> (visited on 05/02/2026).
- [23] Fabio Muratore et al. “Robot Learning from Randomized Simulations: A Review”. In: *CoRR* abs/2111.00956 (2021). arXiv: 2111.00956. URL: <https://arxiv.org/abs/2111.00956>.
- [24] NVIDIA et al. *GR00T N1: An Open Foundation Model for Generalist Humanoid Robots*. 2025. arXiv: 2503.14734 [cs.R0]. URL: <https://arxiv.org/abs/2503.14734>.
- [25] Deepak Pathak et al. *Zero-Shot Visual Imitation*. Apr. 23, 2018. DOI: 10.48550/arXiv.1804.08606. arXiv: 1804.08606 [cs]. URL: <http://arxiv.org/abs/1804.08606> (visited on 05/04/2026). Pre-published.
- [26] Xue Bin Peng et al. “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization”. In: *CoRR* abs/1710.06537 (2017). arXiv: 1710.06537. URL: <http://arxiv.org/abs/1710.06537>.
- [27] “Predictive Control of a Two-Wheeled Balancing Robot: Lab Practice Control Systems - Experiment Script”. In: ().

- [28] Antonin Raffin et al. "DECOUPLING FEATURE EXTRACTION FROM POLICY LEARNING: ASSESSING BENEFITS OF STATE REPRESENTATION LEARNING IN GOAL BASED ROBOTICS". In: (2019).
- [29] Coursera Staff. *What Is a Markov Decision Process?* Coursera. May 15, 2025. URL: <https://www.coursera.org/articles/what-is-a-markov-decision-process> (visited on 04/28/2026).
- [30] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning: an introduction*. Adaptive computation and machine learning. Cambridge, Mass: MIT Press, 1998. 322 pp. ISBN: 978-0-262-19398-6.
- [31] Aviv Tamar et al. "IMITATION LEARNING FROM VISUAL DATA WITH MULTIPLE INTENTIONS". In: (2018).
- [32] Joshua Tobin et al. "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World". In: *CoRR* abs/1703.06907 (2017). arXiv: 1703.06907. URL: <http://arxiv.org/abs/1703.06907>.
- [33] Toshiaki Tsuji et al. "A Survey on Imitation Learning for Contact-Rich Tasks in Robotics". In: *The International Journal of Robotics Research* (Mar. 14, 2026), p. 02783649261417694. ISSN: 0278-3649. DOI: 10.1177/02783649261417694. URL: <https://doi.org/10.1177/02783649261417694> (visited on 05/05/2026).
- [34] Ashish Vaswani et al. "Attention Is All You Need". In: *Neural Information Processing Systems*. 2017. URL: <https://api.semanticscholar.org/CorpusID:13756489>.
- [35] Walkeraastro. *Action Space, State Space, Observation Space demystified*. Medium. Aug. 26, 2024. URL: <https://medium.com/@walkeraastro41/action-space-state-space-observation-space-demystified-6c9c00a355b4> (visited on 04/28/2026).
- [36] Wenshuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. "Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey". In: *CoRR* abs/2009.13303 (2020). arXiv: 2009.13303. URL: <https://arxiv.org/abs/2009.13303>.

List of Figures

3.1	Base concept of Reinforcement Learning	7
3.2	Overview Robotic Learning	8
3.3	Training Progress in Deep Learning and Reinforcement Learning	10
3.4	State-Value Function	11
3.5	State-Action Value Function	12
4.1	Image-based instructions for task execution [2].	18
4.2	Problem: Data Bottleneck in Robotics	26
4.3	Key Challenges in Learning from Videos	30
5.1	Video Foundation Modelling for LfV	34
5.2	To label action-free videos, an Alternative action $\hat{a}$ can be derived from video data instead of the missing Action a	43
5.3	Categories of Alternative Action Representations that can be learned or obtained from video data.	46

List of Tables

5.1	Beispielhafte Key-Frames in einer Pick-and-Place-Sequenz	47
-----	--	----